# Programmazione Lineare Intera

## 1. Problemi di ottimizzazione lineare discreta

All'inizio del corso abbiamo formulato molti problemi, alcuni dei quali hanno comportato l'utilizzo di variabili logiche (a valori 0 o 1) o variabili intere. Questa classe problemi, se funzione obiettivo e vincoli sono lineari, sono indicati con il termine *ottimizzazione lineare discreta*.
Consideriamo un problema detto di *Programmazione Lineare Intera* del tipo:

$$\begin{aligned} \min \quad & cx \\ & Ax \geq b \\ & x \in \mathbf{Z}^n_+ \end{aligned}$$

dove $\mathbf{Z}^n_+$ denota l'insieme dei vettori $n$ dimensionali a componenti intere non negative. In fig. 1 è riportata la rappresentazione geometrica di un tale problema, dove la regione ammissibile è data dai soli punti che cadono sugli incroci della quadrettatura.

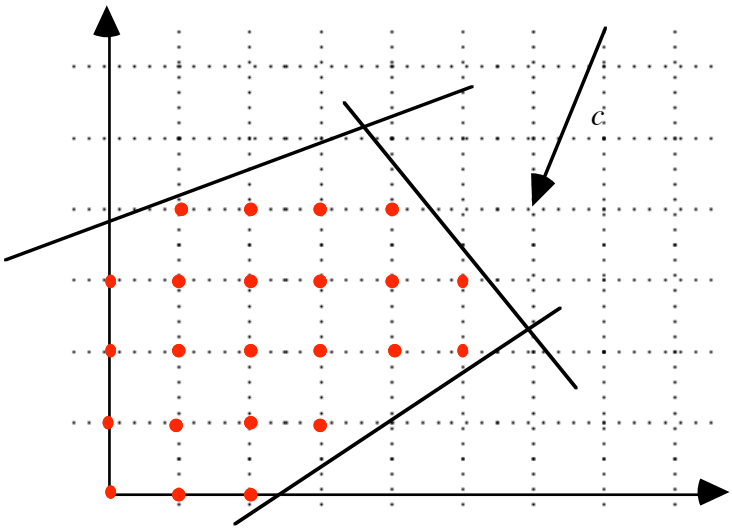


fig. 1: in rosso sono riportati i punti della regione ammissibile

Si noti che la regione ammissibile (costituita da un insieme discreto di punti) <u>non</u> è più un insieme convesso, come succedeva nella programmazione lineare. Ne consegue che la teoria sviluppata per la *PL* e, di conseguenza, gli algoritmi visti non possono essere applicati direttamente a questa classe di problemi.

Oltre ai problemi di Programmazione Lineare Intera, nella classe di problemi di ottimizzazione lineare discreta abbiamo anche i problemi di *Programmazione Lineare Mista* (PLM), cioè la classe dei problemi di PL in cui solo un sottoinsieme di variabili sono a valori interi. Un problema di PLM è ad esempio

$$\begin{aligned} \min \quad & cx + dy \\ & Ax + Dy \geq b \\ & x \in \mathbf{Z}^n_+,\ y \in \mathbf{R}^n \end{aligned}$$

Un caso particolare di Programmazione Lineare Intera è dato dall'*Ottimizzazione Combinatoria* (OC), cioè la classe di problemi in cui la regione ammissibile è un sottoinsieme dei vertici dell'ipercubo unitario $F \subseteq \mathbf{B}^n = \{0,1\}^n$, ovvero in parole povere, in cui le variabili possono assumere solo valori 0 o

1. Appartengono a questa classe i problemi di programmazione lineare {0,1} (o binaria) come ad esempio:

$$\begin{aligned} \min \quad & cx \\ & Ax \geq b \\ & x \in \boldsymbol{B}^n \end{aligned}$$

In precedenza abbiamo visto vari problemi di *OC*, come ad esempio il problema dello zaino, quello del commesso viaggiatore o quello dell'albero di copertura.
I problemi di ottimizzazione discreta sono in generale difficili da risolvere. Tuttavia i problemi di ottimizzazione discreta costituiscono una classe molto importante poiché risultano uno strumento molto efficace per la modellazione di problemi reali, come abbiamo visto in precedenza e come sarà mostrato nel seguito del capitolo.
Vediamo ora alcune formulazioni di problemi di ottimizzazione discreta, a completamente di quanto visto nel capitolo 1.

<u>Esempio: vincoli disgiuntivi</u>
Fino ad ora abbiamo visto esempi di problemi in cui la regione ammissibile è descritta da un poliedro convesso. Potrebbero verificarsi casi in cui tale proprietà viene meno e la regione ammissibile è data da un poliedro non convesso. Consideriamo ad esempio la regione ammissibile descritta in figura 2.

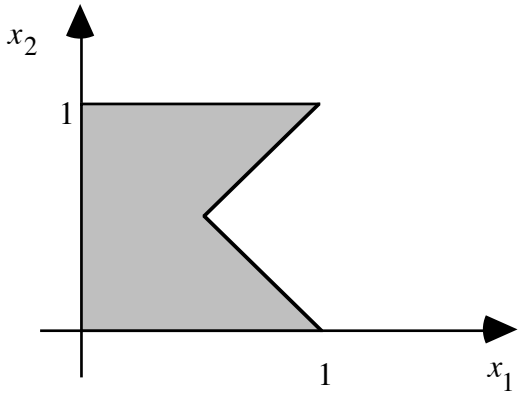


fig. 2 esempio di regione ammissibile non convessa

La regione in figura può essere descritta come unione di due poliedri convessi *P* e *R*, come evidenziato dalla figura 3.

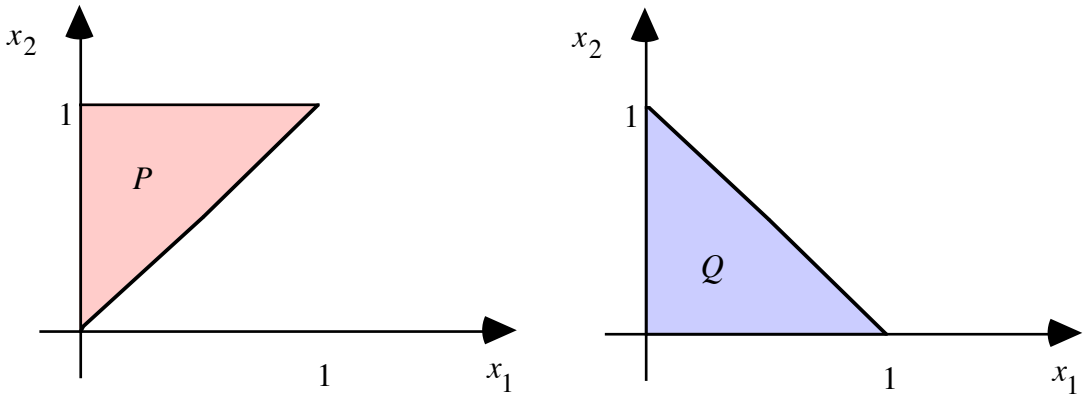


fig. 3: decomposizione del poliedro

Il poliedro $P$ è dato da $P=\{x\in\boldsymbol{R}^2: x_1 - x_2 \leq 0, x_2\leq 1, x_1\geq 0, x_2\geq 0\}$, mentre $Q=\{x\in\boldsymbol{R}^2: x_1 + x_2 \leq 1, x_1\geq 0, x_2\geq 0\}$. Per fare l'intersezione dei due poliedri sappiamo che basta che tutti i vincoli di $P$ e di $Q$ valgano contemporaneamente, se invece ci interessa l'unione dobbiamo imporre che almeno uno dei due gruppi di vincoli valga. In primo luogo osserviamo che alcuni vincoli sono comuni ai due poliedri, in particolare $\{x_2\leq 1, x_1\geq 0, x_2\geq 0\}$, quindi di fatto di questi vincoli possiamo anche fare l'intersezione, mentre quelli che caratterizzano più propriamente le due regioni ammissibili sono $x_1 - x_2 \leq 0$ per quel che riguarda $P$ e $x_1 + x_2 \leq 1$ per quel che riguarda $Q$. Dobbiamo fare in modo che nella nostra formulazione almeno uno di questi due vincoli valga, quindi, considerando il problema dal punto di vista opposto, questo significa che al massimo uno di questi due vincoli può venire reso ridondante. Annullare uno di questi vincoli significa traslarlo di una quantità sufficiente da fargli includere tutta la regione ammissibile dell'altro poliedro, come evidenziato in figura 4.

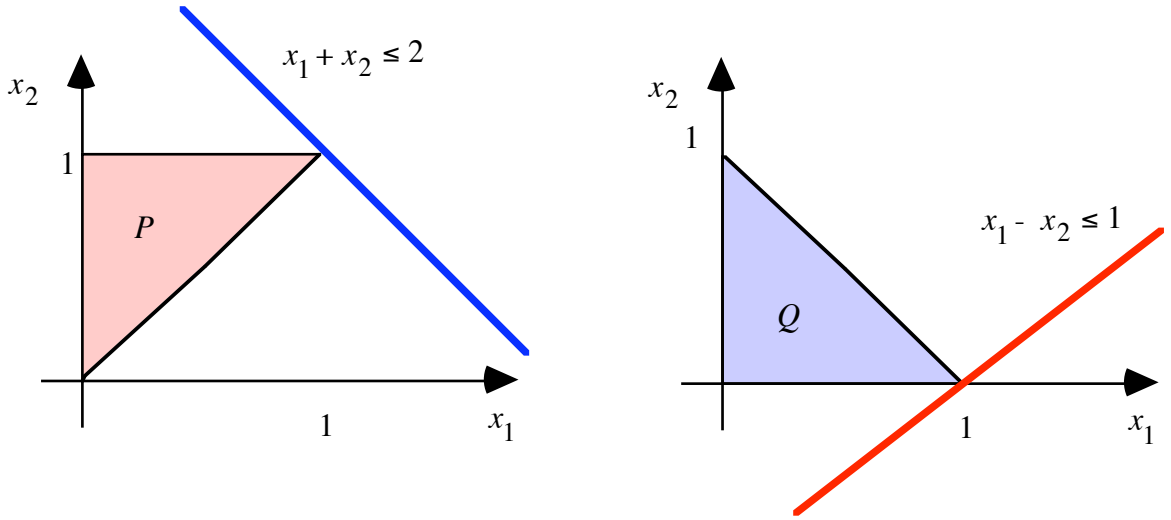


fig. 4: traslazione dei vincoli

Per poter effettuare questa operazione di annullamento di al più un vincolo (o un gruppo di vincoli) tra i due individuati, introduciamo le variabili 0-1 $y_P$ e $y_Q$ che valgono 1 se decidiamo di annullare il vincolo relativo a $P$ o a $Q$, rispettivamente. La formulazione della regione in figura 2 risulta quindi:

$$
\begin{aligned}
&x_2\leq 1\\
&x_1\geq 0\\
&x_2\geq 0\\
&x_1 - x_2 \leq 0 + M_P y_P\\
&x_1 + x_2 \leq 1 + M_Q y_Q\\
&y_P + y_Q \leq 1\\
&y_P, y_Q \in \{0,1\}
\end{aligned}
$$

Il vincolo $y_P + y_Q \leq 1$ consente di annullare al più uno dei due gruppi di vincoli. La scelta delle due costanti $M_P$ e $M_Q$ deve essere tale da garantire che la regione ammissibile sia completamente contenuta dal vincolo traslato. Nel nostro caso quindi $M_P \geq 1$ e $M_Q \geq -1$.

Un caso particolare di applicazione della tecnica dei vincoli disgiuntivi riguarda i problemi di scheduling di lavori su macchine. Poniamoci il problema di processare $n$ lavori $1,\ldots,n$ su $m$ macchine denotate da $1,\ldots, m$. Ogni lavoro deve essere eseguito prima sulla macchina 1, poi sulla 2, e così via fino ad essere eseguito al termine sull'ultima macchina $m$. Il tempo di esecuzione del lavoro $j$ sulla

macchina $k$ è dato da $s_{jk}$, $j$=1,…,$n$ e $k$=1,…,$m$. Le varie macchine possono eseguire un lavoro alla volta e i lavori su ogni macchina devono essere eseguiti fino al termine, cioè non possono essere interrotti e ripresi. Il problema consiste nella determinazione del sequenziamento dei vari lavori sulle diverse macchine con l'obiettivo di minimizzare il tempo di fine dell'ultimo lavoro (in gergo chiamato *makespan*). Per stabilire il sequenziamento abbiamo bisogno di variabili $t_{jk}$ che fissano il tempo di inizio del lavoro $j$ sulla macchina $k$. Con queste variabili è semplice formulare i vincoli che riguardano la sequenza di lavorazione di ogni singolo lavoro:

$$t_{jk} + s_{jk} \le t_{jk+1} \qquad j=1,\dots,n,\ k=1,\dots,m-1$$

il lavoro $j$ non può iniziare ad essere eseguito sulla macchina $k$+1 prima di aver terminato sulla macchina $k$. Per rappresentare correttamente la funzione obiettivo che è di tipo bottleneck (min max) introduciamo una variabile $T$ che sarà maggiore o uguale del tempo di fine dell'ultimo lavoro sull'ultima macchina:

$$t_{jm} + s_{jm} \le T \qquad j=1,\dots,n,$$

Ovviamente la funzione obiettivo tenderà a minimizzare $T$, in modo che nella soluzione ottima $T$ assumerà proprio il tempo di fine dell'ultimo lavoro:

$$\min \quad T$$

Non rimane che formulare i vincoli che impediscono la contemporaneità di esecuzione di due lavori sulla stessa macchina. Questa idea di non contemporaneità è resa considerando ogni coppia di lavori $i$ e $j$ e ogni macchina $k$, e imponendo che sulla macchina $k$ o $i$ precede $j$ o $j$ precede $i$, cioè:

$$t_{ik} + s_{ik} \le t_{jk}$$

oppure

$$t_{jk} + s_{jk} \le t_{ik}$$

Per esprimere la disgiunzione tra i due vincoli abbiamo bisogno di una variabile 0-1 $y_{ijk}$ che vale 1 se $i$ precede $j$ su $k$ e 0 viceversa. I vincoli (disgiuntivi) sono:

$$t_{ik} + s_{ik} \le t_{jk} + M(1-y_{ijk}) \qquad i,j=1,\dots,n,\ (i\neq j),\ k=1,\dots,m$$
$$t_{jk} + s_{jk} \le t_{ik} + My_{ijk} \qquad i,j=1,\dots,n,\ (i\neq j),\ k=1,\dots,m$$

dove $M$ è un valore sufficientemente elevato.

Generalizzando la tecnica dei vincoli disgiuntivi vista per il caso della regione non convessa e per lo scheduling, consideriamo il problema di determinare una soluzione che soddisfi almeno $k$ ($1\le k\le n$) dei vincoli:

$$A_i x \le b_i, \quad i = 1,\dots,n.$$

Sia $M\in\mathbf{R}$: $A_i x - b_i \le M$, per ogni $x$ ammissibile, $i$ = 1,…,$n$.
Il problema può essere formulato come segue:

$$A_i x \leq b_i + My_i, \qquad i = 1,\ldots,n,$$
$$\sum_{i=1}^{n} y_i \leq n - k,$$
$$y \in \{0,1\}^n.$$

**Esercizio**

Formulare come PLI il seguente problema: min $\{cx: x \in X\}$ dove la regione ammissibile $X$ è indicata in fig. 5

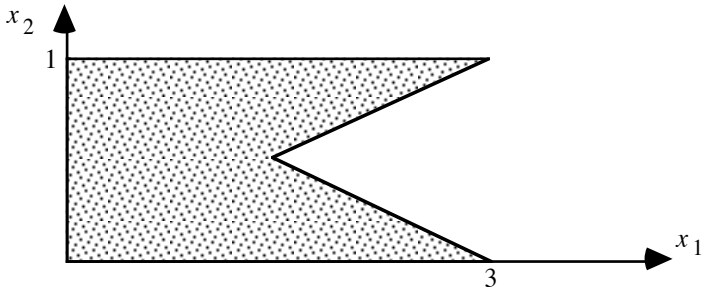


fig. 5

<u>Esempio: funzioni lineari a tratti</u>

Prima dell'avvento della deregolamentazione nel mondo della telefonia italiana vigeva la seguente tariffa per le utenze domestiche: per i primi 140 scatti il costo era di 50 lire, dal 141-esimo scatto al 220-esimo il costo era di 207 lire e per i successivi 127 lire. In figura 6 viene rappresentato uno schema di tale funzione costo, dove $b_0$ rappresenta il costo fisso (per esempio il costo del canone bimestrale).

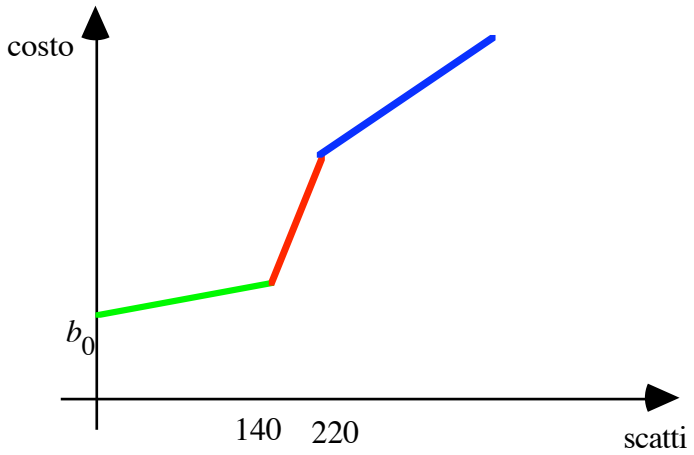


fig. 6: vecchia tariffa sip per utenze domestiche

Quindi, indicando con $x$ il numero di scatti e con $f(x)$ la funzione costo, formalmente abbiamo:

$$f(x) = \begin{cases} 0 & \text{se } x = 0 \\ b_0 + 50x & \text{se } 0 < x \leq 140 \\ b_1 + 207x & \text{se } 140 < x \leq 220 \\ b_2 + 127x & \text{se } 220 < x \end{cases}$$

dove $b_1$ e $b_2$ sono i valori dell'intercetta delle rette rossa e verde in figura. Volendo utilizzare questa funzione costo in un problema di ottimizzazione ci troveremmo davanti alla difficoltà di avere a che fare con una funzione obiettivo non lineare, anche se *lineare a tratti*. Vi è un modo di trattare queste funzioni ricorrendo a modelli di programmazione lineare intera. L'idea è di trattare separatamente i tre intervalli [0,140], (140,220], (220,$M$] con $M$ costante opportunamente grande. Introduciamo delle

variabili continue $z_0$, $z_1$ e $z_2$ che indicano quanti scatti vengono consumati nella prima fascia, nella seconda e nella terza, rispettivamente. Dobbiamo anche introdurre un meccanismo che impedisca consumare gli scatti di un intervallo se prima non sono stati esauriti quelli dell'intervallo precedente. Questo viene fatto introducendo delle variabili 0-1, una per ogni intervallo, ($y_0$, $y_1$ e $y_2$) che valgono 1 solo se abbiamo iniziato a consumare qualche scatto nell'intervallo a cui si riferiscono. La funzione obiettivo e i vincoli che realizzano il meccanismo a cui abbiamo accennato sono:

$$
\begin{array}{lllll}
\min & b_0 y_0 + 50 z_0 + 207 z_1 + 127 z_2 & & & \\
 & 140\, y_1 & \le & z_0 & \le \quad 140\, y_0 \\
 & 80\, y_2 & \le & z_1 & \le \quad 80\, y_1 \\
 & 0 & \le & z_2 & \le \quad M\, y_2 \\
 & x = z_0 + z_1 + z_2 & & & \\
 & y_0, y_1 \text{ e } y_2 \in \{0,1\} & & &
\end{array}
$$

Si noti che se per esempio $z_1>0$ allora $y_1$ deve risultare uguale a 1, il che implica che $z_0$ deve essere uguale a 140, quindi dobbiamo aver esaurito tutti gli scatti dell'intervallo precedente.
Volendo generalizzare quanto visto nel caso dell'esempio della tariffa telefonica ad un contesto più generale, consideriamo la seguente funzione lineare a tratti da minimizzare, la cui rappresentazione grafica è data in figura 7:

$$
f(x)=\begin{cases}
0, & \text{se } x = a_0, \\
b_0 + c_0(x - a_0), & \text{se } a_0 < x \le a_1, \\
b_0 + c_0(a_1 - a_0) + b_1 + c_1(x - a_1), & \text{se } a_1 < x \le a_2, \\
\dots & \dots \\
\sum_{i=1}^{k-1} [b_{i-1} + c_{i-1}(a_i - a_{i-1})] + b_{k-1} + c_{k-1}(x - a_{k-1}), & \text{se } a_{k-1} < x \le a_k.
\end{cases}
$$

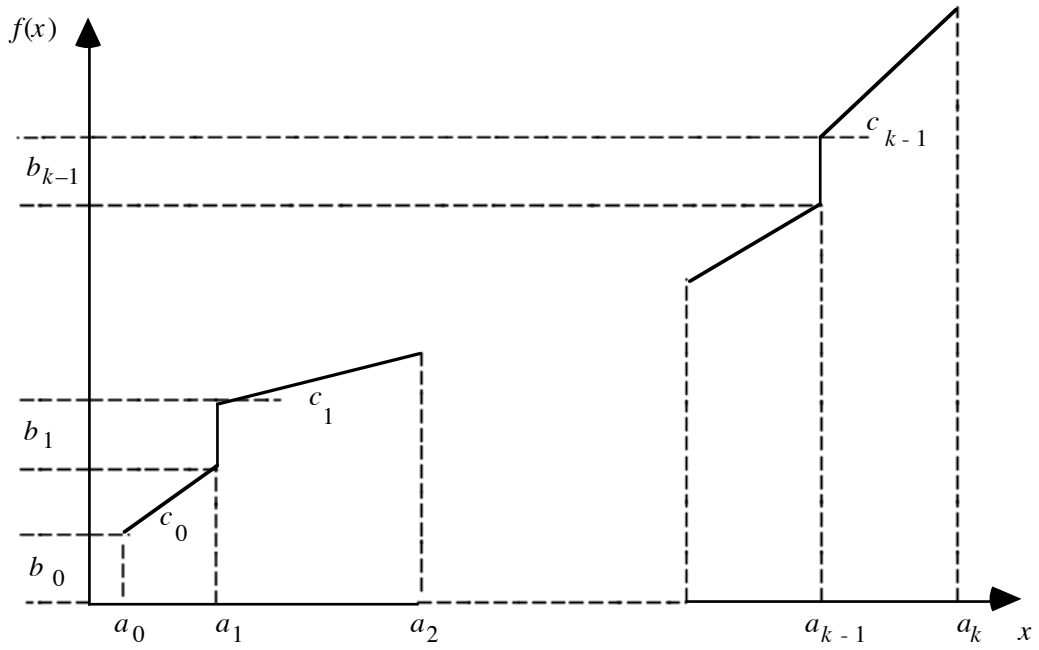


fig. 7: funzione lineare a tratti generale

Il problema $P$ può essere formulato come problema di PLI nel seguente modo:

$$
\begin{array}{lll}
(P') & \min & \sum_{i=0}^{k-1} b_i y_i + c_i z_i \\
 & & (a_1 - a_0)y_1 \le z_0 \le (a_1 - a_0)y_0, \\
 & & (a_2 - a_1)y_2 \le z_1 \le (a_2 - a_1)y_1, \\
 & & \quad \dots \qquad \dots \qquad \dots \\
 & & \qquad 0 \le z_{k-1} \le (a_k - a_{k-1})y_{k-1}, \\
 & & x = \sum_{i=0}^{k-1} z_i + a_0, \\
 & & y_i \in \{0,1\}, z_i \in \boldsymbol{R}, i = 0,\dots, k\text{-}1.
\end{array}
$$

Sia $(\bar{y}, \bar{z}, \bar{x})$ una soluzione ammissibile per $P'$. Vogliamo mostrare che risulta

$$f(\bar{x}) = \sum_{i=0}^{k-1} b_i \bar{y}_i + c_i \bar{z}_i.$$

Consideriamo $\bar{y}$. Assumiamo che risulti $\bar{y}_{r-1} = 1$, $\bar{y}_r = 0$, $1 \le r \le k\text{-}1$.

$$\bar{y}_r = 0 \quad \Rightarrow \quad \begin{cases} \bar{y}_i = 0, & i \ge r, \\ \bar{z}_i = 0 & i \ge r; \end{cases}$$

$$\bar{y}_{r-1} = 1 \Rightarrow \quad \begin{cases} \bar{y}_i = 1, & i \le r\text{-}1, \\ \bar{z}_{i-1} = a_i - a_{i-1} & i \le r\text{-}1, \\ 0 \le \bar{z}_{i-1} \le a_i - a_{i-1}, & i = r; \end{cases}$$

Pertanto:

$$
\begin{aligned}
\sum_{i=0}^{k-1} b_i \bar{y}_i + c_i \bar{z}_i &= \sum_{i=0}^{r-1} b_i + \sum_{i=1}^{r-1} c_{i-1}(a_i - a_{i-1}) + c_{r-1} \bar{z}_{r-1} \\
&= \sum_{i=0}^{r-1} b_i + \sum_{i=1}^{r-1} c_{i-1}(a_i - a_{i-1}) + c_{r-1}(\bar{x} - a_{r-1}).
\end{aligned}
$$

L'ultima uguaglianza deriva dal fatto che per costruzione è:

$$\bar{x} = a_{r-1} + \bar{z}_{r-1} \text{ con } 0 \le \bar{z}_{r-1} \le (a_r - a_{r-1}),$$

e quindi $f(\bar{x}) = \sum_{i=0}^{k-1} b_i \bar{y}_i + c_i \bar{z}_i$.

Vi sono situazioni particolari in cui non è necessario introdurre variabili 0,1 per indurre il meccanismo di sequenzialità nella crescita delle variabili $z$. Uno di questi è per esempio la minimizzazione di una funzione lineare a tratti convessa, di cui riportiamo un esempio in figura 8.

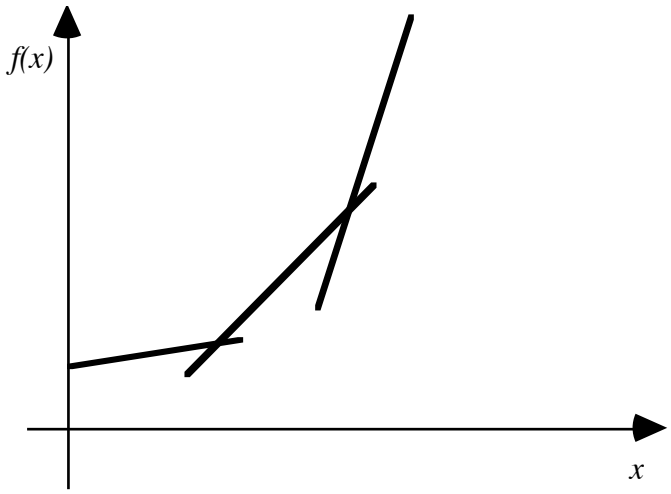


fig.8: funzione convessa lineare a tratti

Con una simile funzione è abbastanza semplice intuire che vi è maggior vantaggio a utilizzare la parte iniziale della funzione, con costi unitari più bassi, prima di passare a quelle successive. Formalmente possiamo vedere che tali funzioni possono venire rappresentate con la tecnica delle funzioni obiettivo di tipo bottleneck vista nel primo capitolo. Sia data la funzione convessa lineare a tratti:

$$f(x) = \begin{cases} b_0 + c_0 x & \text{se } 0 \le x \le a_0 \\ b_1 + c_1 x & \text{se } a_0 < x \le a_1 \\ \dots & \dots \\ b_n + c_n x & \text{se } a_{n-1} < x \end{cases}$$

Se consideriamo le varie rette anche al di fuori del campo di definizione degli intervalli, vediamo che la funzione è data sempre dalla retta più in alto, quindi:

$$f(x) = \max\{b_i + c_i x,\ i=0,\dots n-1\}.$$

**Esercizio**
Si consideri una funzione obiettivo $f(x)$ concava lineare a tratti. Si formuli il problema di massimizzare $f(x)$ in termini di programmazione lineare.

<u>Esempio: Il problema del commesso viaggiatore su grafo orientato</u>
Si consideri il problema del commesso viaggiatore ovvero il problema di determinare un ciclo Hamiltoniano di costo minimo su un grafo simmetrico pesato $G=(N,A,c)$ dove $|N| = n$. Nel capitolo 2 avevamo visto una possibile formulazione del problema del commesso viaggiatore su grafi non orientati, analizziamo ora il caso orientato.
Sia $x = [x_{ij}]$ un vettore di variabili binarie con il seguente significato:

$$x_{ij} = \begin{cases} 1, & \text{se l'arco } (i,j) \text{ appartiene al ciclo Hamiltoniano cercato,} \\ 0, & \text{altrimenti.} \end{cases}$$

Un primo insieme di vincoli ci dice che in ogni nodo deve arrivare esattamente un arco e ne deve partire esattamente uno:

$$X_1 = \{x: \sum_{i=1}^{n} x_{ij} = 1,\ \sum_{j=1}^{n} x_{ij} = 1,\ x_{ij} \in \{0,1\},\ i = 1,\dots, n,\ j = 1,\dots, n\}.$$

Come nel caso non orientato dobbiamo eliminare i sottocicli, cioè i cicli che non toccano tutti i nodi del grafo. Un primo modo è il seguente

$$X_2 = \{x: \sum_{i \in N_1} \sum_{j \in N_2} x_{ij} \geq 1, \forall\ (N_1, N_2,) : N = N_1 \cup N_2, N_1 \cap N_2 = \emptyset, N_1 \neq \emptyset, N_2 \neq \emptyset\},$$

Considerata una qualsiasi partizione di nodi, in due sottoinsiemi non vuoti, il numero di archi in soluzione che ha la coda nel primo sottoinsieme e la testa nel secondo deve essere maggiore o uguale a 1. Un modo alternativo è quello che introduce altre variabili (continue) $u_i$ che danno l'ordine di visita di ciascun nodo $i$. I vincoli servono a definire il valore delle variabili che danno l'ordinamento.

$$X_3 = \{x: u_i - u_j + nx_{ij} \leq n\text{-}1\ ,\ u_i \in \boldsymbol{R},\ i\ = 1,\ldots, n,\ j = 2,\ldots n,\ i\ \neq j\ \}.$$

L'insieme $X_3$ include i soli cicli che contengono il nodo 1. Infatti consideriamo un ciclo costituito da $k \leq n$-1 nodi che non contenga il nodo 1; sommando i vincoli $u_i - u_j + nx_{ij} \leq n$-1 che corrispondono agli archi del ciclo si ottiene l'assurdo $kn \leq k\ (n\text{-}1)$. Si osservi che $X_3$ <u>non</u> include i vincoli $u_i - u_1 + nx_{ij} \leq n$-1, $u_i \in \boldsymbol{R}$, $i\ = 1,\ldots, n$ e pertanto è soddisfatto dai cicli che contengono il nodo 1. Di conseguenza l'insieme $X_1 \cap X_3$ definisce l'insieme dei cicli Hamiltoniani su $G$.
Vediamo ora che le variabili $u_i$ forniscono l'ordine di visita in una soluzione ammissibile.
Se l'arco $(i,j)$ appartiene al ciclo Hamiltoniano ($x_{ij} = 1$) il corrispondente vincolo in $X_3$ diventa:

$$u_i + 1 \leq u_j.$$

Si osservi che un vettore $\bar{u}$ che soddisfa i precedenti vincoli è data da $\bar{u}_1 = 0$ , $\bar{u}_i = k$, $i \neq 1$, se $i$ è il $k$+1-esimo nodo del ciclo a partire dal nodo 1.
Se l'arco $(i,j)$ non appartiene al ciclo Hamiltoniano ($x_{ij} = 0$) il corrispondente vincolo in $X_3$ diventa:

$$u_i - u_j \leq n\text{-}1,$$

che è sempre soddisfatto da $\bar{u}$, in quanto $\bar{u}_i \leq n$ -1 e $\bar{u}_j \geq 0$.
Il problema del commesso viaggiatore può essere definito in uno dei due modi alternativi

$$\text{TSP1:} \qquad \min \{ \sum_{i,j=1}^{n} c_{ij}x_{ij} : x \in X_1 \cap X_2\};$$

$$\text{TSP2:} \qquad \min \{ \sum_{i,j=1}^{n} c_{ij}x_{ij} : x \in X_1 \cap X_3\}.$$

Si osservi che TSP1 è definito da $O(2^n)$ vincoli, mentre TSP2 è definito da $O(n^2)$ vincoli.

<u>Esempio: selezione di sottoinsiemi</u>
Molti problemi di rilevanza pratica vengono formulati come selezione di sottoinsiemi appartenenti a una determinata famiglia. A questa classe di problemi appartengono per esempio i problemi di turnazione di personale.
Siano dati:
- un insieme $I = \{1, 2,\ldots, n\}$,
- una famiglia $F$ di $m$ sottoinsiemi di $I$ : $F = \{J_1, J_2,\ldots, J_m\}$, $J_j \subseteq I$ , $j = 1,\ldots, m$,
- il costo $c_j$ del sottoinsieme $J_j$, $j = 1,\ldots, m$.

Si vuole determinare una famiglia $D \subseteq F$ di costo minimo, con il vincolo che ogni elemento di $I$ appartenga a:

i) almeno un sottoinsieme di $D$ (problema di *copertura*);
ii) esattamente un sottoinsieme di $D$ (problema di *partizione*);
iii) al più un sottoinsieme di $D$ (problema di *riempimento*).

Per esempio, nel caso di turnazione dei piloti di una compagnia aerea, l'insieme $I$ è dato dai voli da coprire, mentre ogni sottoinsieme $J_j$ è definito dai voli che possono venire eseguiti da uno stesso pilota in una settimana lavorativa soddisfacendo le regole sindacale e la compatibilità tra voli dettata dai semplici vincoli temporali e dal tipo di aeromobile. Infine $c_j$ è la paga dovuta al pilota per quel determinato turno di lavoro. Il problema è quello di partizione (dobbiamo assegnare un pilota a ogni volo), ma siccome spesso è difficile ottenere una soluzione ammissibile si può considerare il problema di copertura, correndo il rischio di avere una soluzione con più piloti su uno stesso volo.

Formulando i problemi descritti sopra in termini di Programmazione Lineare Intera, la famiglia $F$ può essere rappresentata dalla matrice $A = [a_{ij}]$, $i = 1,\ldots, n$, $j = 1,\ldots, m$, dove:

$$a_{ij} = \begin{cases} 1, & \text{se } i \in J_j, \\ 0, & \text{altrimenti.} \end{cases}$$

Se indichiamo le variabili di scelta dei sottoinsiemi con

$$x_j = \begin{cases} 1, & \text{se l'insieme } J_j \in D, \\ 0, & \text{altrimenti,} \end{cases}$$

allora:

- il problema di *copertura* può essere formulato come:

$$\begin{aligned} \min \quad & cx \\ & Ax \geq e \\ & x \in \{0,1\}^m \end{aligned}$$

  dove $e$ denota il vettore le cui componenti sono uguali a 1;
- il problema di *partizione* può essere formulato come:

$$\begin{aligned} \min \quad & cx \\ & Ax = e \\ & x \in \{0,1\}^m; \end{aligned}$$

- il problema di *riempimento* può essere formulato come:

$$\begin{aligned} \min \quad & cx \\ & Ax \leq e \\ & x \in \{0,1\}^m. \end{aligned}$$

<u>Esempio: instradamento di veicoli</u>

Un classico problema di ottimizzazione nei trasporti è quello dell'instradamento di veicoli (*Vehicle Routing Problem*). Sono dati un insieme di clienti da servire $\{1,\ldots,n\}$, ognuno dei quali ha una richiesta di merce $d_i$, un insieme di $k$ veicoli che, per semplicità, assumiamo essere omogenei, di uguale capacità $Q$ e tutti ospitati nel medesimo deposito, che identifichiamo con 0. Sono note anche le distanze tra ogni coppia di clienti e tra il deposito e ogni cliente ($c_{ij}$, $i,j = 0,\ldots,n$). Il problema consiste nel consegnare la merce a tutti i clienti minimizzando la distanza complessiva percorsa dai veicoli e

rispettando i vincoli di capacità degli stessi. A supporto della formulazione possiamo considerare un grafo completo con nodi corrispondenti ai clienti e al deposito. Una soluzione ammissibile consiste in una famiglia di $m$ ($m \leq k$) cicli con origine in 0 tali che tutti i clienti appartengono ad un ciclo. Possiamo decomporre le decisioni in due livelli: in un primo livello assegnamo i clienti ai veicoli. Utilizziamo delle variabili di assegnamento

$$y_{ih} = \begin{cases} 1, & \text{se il nodo } i \text{ appartiene all'itinerario del veicolo } h, \\ 0, & \text{altrimenti,} \end{cases}$$

I vincoli che si riferiscono a queste variabili sono:

$$\sum_{h=1}^{k} y_{ih} = 1 \qquad i=1,\ldots,n$$

ovvero, ogni cliente deve appartenere ad un itinerario. Possiamo anche avere un limite sul numero dei veicoli utilizzati mettendo un vincolo sul numero di volte che il nodo 0 viene assegnato ad un itinerario:

$$\sum_{h=1}^{k} y_{1h} \leq m$$

È anche facile esprimere i vincoli sulla capacità dei veicoli imponendo che la somma delle domande dei clienti assegnati ad uno stesso veicolo non ecceda la capacità massima, utilizzando i vincoli che abbiamo visto per il problema dello zaino:

$$\sum_{i=1}^{n} d_i\, y_{ih} \leq Q \quad h=1,\ldots,k$$

Per misurare ora la distanza percorsa da ogni veicolo dobbiamo risolvere tanti problemi del commesso viaggiatore quanti sono i veicoli utilizzati. Introduciamo quindi delle variabili $x_{ij}^h$ per determinare l'itinerario del veicolo $h$:

$$x_{ij}^h = \begin{cases} 1, & \text{se il veicolo } h \text{ va dal nodo } i \text{ al nodo } j, \\ 0, & \text{altrimenti,} \end{cases} \qquad i, j = 0,\ldots,n,\ h = 1,\ldots,k.$$

I vincoli per l'ammissibililtà nella scelta degli itinerari (un arco entrante e uno uscente per ogni nodo e eliminazione di sottocicli) sono:

$$\sum_{j=1, j\neq i}^{n} x_{ij}^h = y_{ih} \qquad i = 0,\ldots,n,\ h=1,\ldots,k$$

$$\sum_{j=1, j\neq i}^{n} x_{ji}^h = y_{ih} \qquad i = 0,\ldots,n,\ h=1,\ldots,k$$

$$\sum_{i,j\in S} x_{ij}^h \leq |S|-1 \qquad S\subseteq\{1,\ldots,n\}\ |S|\geq 2,\ h=1,\ldots,k.$$

Possiamo arrivare ad eliminare un gruppo di variabili e a semplificare il modello osservando che

$$y_{ih} = \sum_{j=1, j\neq i}^{n} x_{ij}^h$$

La formulazione completa del problema di instradamento di veicoli diventa:

$$\min \quad \sum_{h=1}^{k} \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}^{h}$$

$$\sum_{j=1}^{n} x_{ij}^{h} = \sum_{j=1}^{n} x_{ji}^{h} = y_{ih} \qquad i = 0,\ldots,n,\ h=1,\ldots,k$$

$$\sum_{h=1}^{k} \sum_{j=1}^{n} x_{ij}^{h} = 1 \qquad i = 1,\ldots,n,$$

$$\sum_{i=1}^{n} \sum_{j=0}^{n} d_i x_{ji}^{h} \le Q \qquad h=1,\ldots,k.$$

$$x_{ij}^{h} \in \{0,1\}, \qquad i,j = 0,\ldots,n,$$

<u>Esempio: assegnamento di frequenze</u>

In una rete di telecomunicazione cellulare il territorio è servito da varie antenne, ognuna delle quali copre una determinata *cella*. Le celle si sovrappongono tra loro per garantire una continuità del servizio agli utenti in movimento. Un problema classico in questo ambito è quello di assegnare delle frequenze alle celle in modo da evitare che celle interferenti (per esempio quelle che si sovrappongono) utilizzino la stessa frequenza. Uno degli obiettivi può essere per esempio quello di minimizzare il numero di frequenze utilizzate.

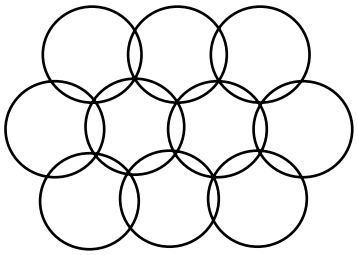

fig. 9: sistema cellulare

Il problema fa ricorso a una rappresentazione su grafo. Costruiamo un grafo non orientato $G = (N,A)$ in cui i nodi corrispondono alle celle della rete di telecomunicazione e vi è un arco $\{i,j\}$ tra due nodi se le corrispondenti celle sono interferenti.

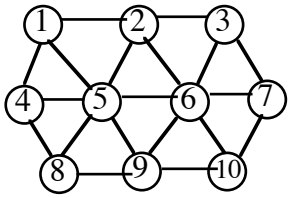


fig. 10: il grafo delle interferenze

Avendo $k$ frequenze a disposizione, per formulare il problema utilizziamo delle variabili di assegnamento dei nodi alle frequenze:

$$x_{ih} = \begin{cases} 1, & \text{se al nodo } i \text{ viene assegnata la frequenza } h, \\ 0, & \text{altrimenti,} \end{cases}$$

Inoltre abbiamo anche bisogno di un insieme di variabili che stabiliscono l'utilizzo effettivo delle frequenze.

$$y_h = \begin{cases} 1, & \text{se la frequenza } h \text{ è assegnata a qualche nodo,} \\ 0, & \text{altrimenti,} \end{cases}$$

La formulazione del problema è:

$$\begin{aligned} \min \quad & \sum_{h=1}^{k} y_h \\ & x_{ih} + x_{jh} \le 1 \quad \forall \{i,j\} \in A \\ & x_{ih}, y_h, \forall i \in N, h=1,\ldots,k. \end{aligned}$$

Questo problema su grafi è noto in letteratura come il problema della *colorazione*: si vogliono colorare i nodi di un grafo in modo che nodi adiacenti abbiano colori diversi, minimizzando il numero di colori. Molto noto il caso della colorazione delle carte geografiche.

## 2. Scelta della formulazione e metodi di soluzione "poliedrali"

A differenza della programmazione lineare continua, nella PLI la scelta della formulazione ha una grande influenza sull'efficienza dei metodi di soluzione e, molto spesso, risulta essere decisiva. Prendiamo in considerazione il seguente esempio.

$$\begin{aligned} \max \quad & x_1 + 0.64\, x_2 \\ & 50x_1 + 31x_2 \le 250 \\ & 3x_1 - 2\, x_2 \ge -4 \\ & x_1, x_2 \ge 0 \text{ interi.} \end{aligned}$$

In figura 11 riportiamo la regione ammissibile data dai punti interi contenuti nel poliedro definito dai vincoli lineari.

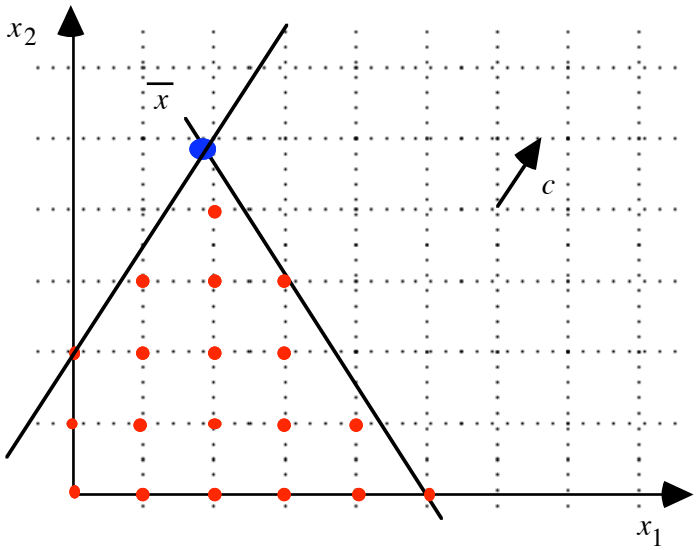


fig. 11: regione ammissibile e soluzione ottima del problema continuo

Nella figura abbiamo riportato anche la soluzione $\bar{x}$ del problema di programmazione lineare ottenuto eliminando i vincoli di interezza sulle variabili. Tale problema può venire risolto agevolmente con i metodi visti nel capitolo precedente e risulta $\bar{x}=\begin{bmatrix} 376/193 \\ 950/193 \end{bmatrix}$, di valore 5.098. Non avendo componenti intere, tale soluzione non può essere considerata come soluzione del problema di PLI originale in quanto non è ammissibile. Potremmo cercare di ottenere una soluzione ammissibile, possibilmente

ottima, approssimando i valori frazionari a valori interi più vicini ai valori delle componenti di $\bar{x}$. Nel nostro caso, guardando il disegno, tale soluzione potrebbe essere $x'=\begin{bmatrix}2\\4\end{bmatrix}$. Si noti come la generazione di tale punto non sia così semplice in generale, cioè quando lo spazio delle soluzioni ha dimensione molto superiore a due. Ad ogni modo, la soluzione così ottenuta, oltre ad essere non facile da generare, risulta anche essere in alcuni casi molto lontana dalla soluzione ottima del problema di PLI. Nel caso del nostro esempio, la soluzione ottima intera è $x^*=\begin{bmatrix}5\\0\end{bmatrix}$, come è facile verificare confrontando i valori della funzione obiettivo.
Cosa sarebbe successo se, invece di utilizzare la formulazione con due vincoli data sopra, avessimo utilizzato la seguente, la cui rappresentazione geometrica è riportata in figura 12:

$$
\begin{array}{lrcl}
\max & x_1 + 0.64\, x_2 & & \\
 & -x_1 + x_2 & \leq & 2 \\
 & x_1 + x_2 & \leq & 6 \\
 & 3x_1 + 2\, x_2 & \leq & 15 \\
 & x_1, x_2 \geq 0 \text{ interi.} & &
\end{array}
$$

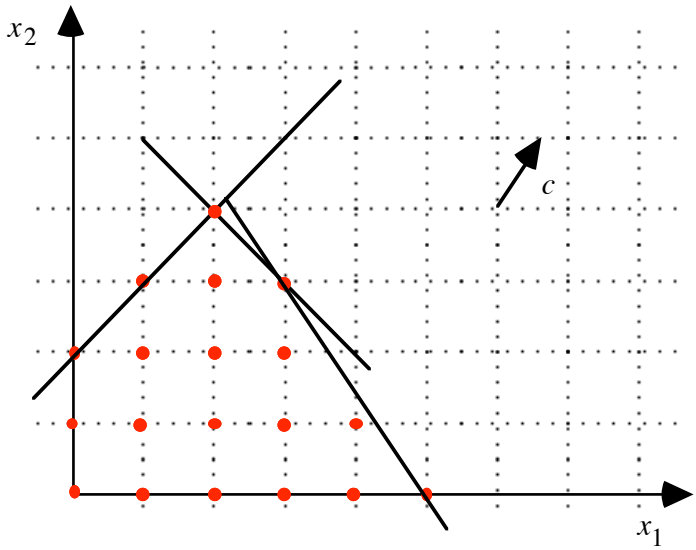


fig. 12: rappresentazione geometrica della formulazione alternativa

Possiamo notare immediatamente che dal punto di vista della regione ammissibile le due formulazioni sono equivalenti, infatti racchiudono lo stesso insieme di punti interi. La seconda formulazione però ha tutti i vertici in corrispondenza a soluzioni ammissibili per il problema di *PLI*. Ricordando il fatto che in un problema di programmazione lineare se l'ottimo esiste finito almeno un vertice del poliedro è ottimo, questo implica che se ignoriamo i vincoli di interezza e risolviamo il corrispondente problema di *PL*, la soluzione che otteniamo è intera e quindi è ottima anche per il problema originale.

Le considerazioni sviluppate nell'esempio precedente mettono in luce come sia molto importante individuare una buona formulazione del problema. Sia $S$ l'insieme delle soluzioni ammissibili di un determinato problema di ottimizzazione discreta, possiamo individuare il più piccolo insieme convesso che include tutti i punti di $S$:

**Definizione 2.1**
Dato un insieme finito di punti $S$, si chiama *inviluppo convesso di S*, indicato con $conv(S)$, il più piccolo poliedro convesso che contiene tutti i punti di $S$.

Consideriamo i problemi:

$$P: \min\ cx \quad Ax = b \quad x \in \mathbf{Z}^n \qquad\qquad P': \min\ cx \quad x \in conv(\{Ax=b,\ x \in \mathbf{Z}^n\})$$

È abbastanza intuitivo convincersi che ogni vertice dell'inviluppo convesso è un punto di $S$, quindi la soluzione ottima di $P'$ è ottima anche per $P$, a causa della linearità della funzione obiettivo. Mentre però non sappiamo ancora come risolvere il problema $P$, $P'$ può essere risolto facendo ricorso agli algoritmi per la programmazione lineare. Poiché se esiste una soluzione ottima finita, almeno un vertice del poliedro coincide con una soluzione ottima, tale vertice appartiene a $S$. In base a queste osservazioni potrebbe sembrare semplice risolvere un problema di ottimizzazione lineare discreta o di *PLI*: si ottiene la formulazione dell'inviluppo convesso in termini di vincoli lineari e si applica l'algoritmo di soluzione dei problemi di *PL* che sappiamo essere efficiente. La difficoltà, ad ogni modo, si nasconde nella formulazione dell'inviluppo convesso. Infatti, per molti problemi, come ad esempio quello del commesso viaggiatore, non è possibile dare la formulazione completa per istanze di dimensioni qualsiasi, mentre in generale tale formulazione, anche quando è possibile caratterizzarla completamente, richiede un numero esponenziale di vincoli, rendendo quindi il metodo di soluzione estremamente inefficiente.

### **2.1 Unimodularità*

Vi sono casi in cui il poliedro $\overline{P}$:$\{Ax = b,\ x \geq 0\}$, in cui abbiamo eliminato i vincoli di interezza, coincide con l'inviluppo convesso $P'=conv(\{Ax=b,\ x \in \mathbf{Z}^n\})$. Questo succede quando ogni vertice di $\overline{P}$, quindi quando ogni sua base, ha componenti intere, ovvero gode della proprietà di *integralità*. Potrebbe essere estremamente utile caratterizzare questi problemi, che risultano particolarmente facili da affrontare. Introduciamo alcune definizioni utili a questo scopo.

**Definizione 2.2**
Una matrice a componenti intere $m \times n$, $A$ , è *unimodulare* se per ogni sottomatrice quadrata non singolare $D$ di rango massimo risulta:

$$\det(D) = \pm 1.$$

**Definizione 2.3**
Una matrice a componenti intere $m \times n$, $A$, è *totalmente unimodulare* (TU) se ogni sua matrice quadrata non singolare è *unimodulare*.

**Teorema 2.4**
Se $A$ e $b$ hanno componenti intere e $A$ è unimodulare, allora tutte le soluzioni di base del problema

$$\min\ cx \quad Ax = b \quad x \geq 0$$

hanno componenti intere.

Dim.
Infatti consideriamo una base $B$ e sia $A_B^*$ la matrice aggiunta di $A_B$. Si ha che

$$x_B = A_B^{-1} b = \frac{A_B^*}{\det(A_B)} b,$$

e quindi la soluzione di base ha componenti intere.♦

**Corollario 2.5**
Se $A$ e $b$ hanno componenti intere e $A$ è totalmente unimodulare, allora i problemi:

$$\begin{array}{ll} \min & cx \\ & Ax \geq b \\ & x \geq 0 \end{array} \qquad\qquad \begin{array}{ll} \min & cx \\ & Ax \leq b \\ & x \geq 0 \end{array}$$

hanno tutte le soluzioni di base a componenti intere.

Dim.
La dimostrazione segue immediatamente, osservando che se $A$ è TU allora anche $[A,I]$, $[A,-I]$ sono TU.♦

La caratterizzazione delle matrici totalmente unimodulari è stato oggetto di studi approfonditi. Riportiamo una condizione sufficiente di unimodularità che oltre ad essere di elegante dimostrazione, ha conseguenze dirette su una vasta classe di problemi di ottimizzazione, come quelli su rete di flusso.

**Teorema 2.6**
Una matrice $A$ con elementi 0, ±1 è TU se valgono le seguenti condizioni:

a) ogni colonna contiene al più due elementi non nulli;
b) gli indici $\{1,\ldots,m\}$ delle righe di $A$ possono essere partizionati in due insiemi $I_1$, $I_2$ tali che:
   - se una colonna ha due elementi dello stesso segno gli indici delle righe corrispondenti non appartengono allo stesso insieme;
   - se una colonna ha due elementi di segno opposto gli indici delle righe corrispondenti appartengono allo stesso insieme.

Dim.
Sia $C$ una sottomatrice quadrata di $A$ di ordine $h$. La dimostrazione è effettuata per induzione sull'indice $h$. Se $h=1$, si ha per definizione $\det(C)=0, \pm 1$.
Sia $1<h\leq n$. Si hanno i seguenti casi esaustivi:

i) o $C$ contiene almeno una colonna nulla e quindi $\det(C)=0$;
ii) o $C$ contiene almeno una colonna con un solo elemento non nullo; in tal caso sviluppando il determinante a partire da una di queste colonne e sfruttando l'ipotesi induttiva si ha che $\det(C)=0, \pm 1$;
iii) oppure ogni colonna di $C$ ha esattamente due elementi diversi da zero; si ha quindi, per la colonna $j$-esima:

$$\sum_{i \in I_1} c_{ij} = \sum_{i \in I_2} c_{ij};$$

infatti, elementi di segno opposto della colonna $j$-esima appartengono allo stesso insieme e quindi si elidono nella somma, mentre elementi uguali appartengono a insiemi diversi. Esiste quindi una combinazione lineare delle righe che fornisce il vettore nullo:

$$\sum_{i \in I_1} C_i - \sum_{i \in I_2} C_i = 0.$$

Quindi det($C$) =0.♦

Come conseguenza del teorema 2.6 si ha che la matrice di incidenza di un grafo orientato è TU ed anche la matrice di incidenza di un grafo non orientato bipartito è TU. Pertanto i problemi di flusso, di cammino minimo, di flusso massimo, di assegnamento bipartito e di trasporto hanno soluzioni di base intere. Ricordiamo che avevamo osservato informalmente questa proprietà durante lo studio dei problemi di flusso su rete e degli algoritmi di soluzione. Si osservi che in generale la matrice di un grafo non orientato non è TU. Infatti un grafo non orientato non è bipartito se contiene almeno un ciclo dispari. Sia $C$ la matrice di incidenza di un ciclo dispari. È immediato verificare che det($C$) = ±2.

## *2.2 Metodi "poliedrali" o dei piani di taglio*

Riprendendo l'idea di ricondursi ad un caso di *PL* formulando il problema in termini di inviluppo convesso, notiamo che non è necessario descrivere completamente il poliedro dell'inviluppo convesso, possiamo piuttosto concentrarci nei dintorni della soluzione ottima. I metodi *poliedrali* o di *piani di taglio* non fanno che raffinare iterativamente il poliedro dell'inviluppo convesso dove pensiamo che si trovi la soluzione ottima. Consideriamo il seguente problema di programmazione lineare intera:

$$\begin{aligned} P\colon \ \min \ & cx \\ & Ax \le b \\ & x \in \mathbf{Z}^n_+ \end{aligned}$$

ed il problema in cui trascuriamo i vincoli di interezza:

$$\begin{aligned} \overline{P}\colon \ \min \ & cx \\ & Ax \le b \\ & x \ge 0 \end{aligned}$$

$\overline{P}$ è un problema di programmazione lineare che supponiamo ammetta soluzione ottima finita $\overline{x}$. Se $\overline{x}$ ha tutte componenti intere allora coincide con la soluzione ottima del problema intero $P$, altrimenti possiamo cercare di raffinare la formulazione di $\overline{P}$ aggiungendo opportuni vincoli.

**Definizione 2.7**

Una *disuguaglianza valida* è un vincolo $gx \ge \gamma$, tale che:

$$gx \le \gamma, \qquad \forall x \in \{x \in \mathbf{Z}^n_+ \colon Ax \le b\}$$

**Definizione 2.8**

Un *piano di taglio* è una disuguaglianza valida $gx \le \gamma$, tale che:

$$g\overline{x} > \gamma.$$

Pertanto, se individuiamo un piano di taglio per il problema $\overline{P}$ e la sua soluzione ottima frazionaria $\overline{x}$, possiamo aggiungere tale disuguaglianza alla formulazione e iterare la procedura avvicinandoci progressivamente alla soluzione ottima intera. Il procedimento è riassunto dalla seguente procedura:

```
Procedure metodo_piani_di_taglio:
  begin
    ottimo:= false;
    repeat
      Risolvi(P̄,x̄);
      if x̄ ha tutte componenti intere then ottimo:= true
        else begin
             Genera_taglio (g, γ);
             Aggiungi_taglio(P̄,g, γ)
           end
    until ottimo
  end.
```

fig. 13: Metodo dei piani di taglio

Il punto chiave risulta quindi trovare un metodo che ci permetta di generare dei piani di taglio il più efficaci possibile, cioè vicini al poliedro dell'inviluppo convesso. Descriviamo un metodo utile a ricavare delle disuguaglianze valide, dovuto a Chvátal, che però non sono necessariamente piani di taglio per la soluzione frazionaria.
Consideriamo il problema di programmazione intera

$$(P) \qquad \min\{cx : x \in F\},$$

dove $F = \{x: Ax \le b,\ x \in \mathbf{Z}^n_+\}$, con $A$ matrice $m \times n$ a componenti intere non negative e $b$ vettore colonna a $m$ componenti intere e $d$ a componenti intere non negative.

**Teorema 2.9**
Dato un vettore $u=[u_i] \ge 0$ $i=1,\ldots,m$, ogni disuguaglianza $gx \le \gamma$ con

$$(2.1) \qquad \begin{aligned} g_j &= \lfloor \sum_{i=1}^{m} u_i A_{ij} \rfloor, \\ \gamma &= \lfloor \sum_{i=1}^{m} u_i b_i \rfloor, \end{aligned}$$

è una disuguaglianza valida per $P$.

<u>Dim.</u>
Chiamiamo $\overline{F} = \{x: Ax \le b,\ x \in \mathbf{R}^n_+\}$ e consideriamo il vincolo:

$$(2.2) \qquad \sum_{j=1}^{n} (\sum_{i=1}^{m} u_i A_{ij}) x_j \le \sum_{i=1}^{m} u_i b_i \ .$$

Poiché è stato ottenuto come combinazione lineare non negativa (con moltiplicatori $u_i$) dei vincoli $Ax \le b$, tale vincolo è soddisfatto da ogni $x \in \overline{F}$ e quindi anche da ogni $x \in F$.
Consideriamo adesso il vincolo

$$(2.3) \qquad \sum_{j=1}^{n} g_j x_j \le \sum_{i=1}^{m} u_i b_i \ .$$

Poiché $x \geq 0$, il vincolo in esame è meno stringente rispetto a (2.2), avendo sostituito ai coefficienti originali la loro parte intera inferiore; quindi è soddisfatto da ogni $x \in F$. Poiché per $x \in F$ il lato sinistro di (2.3) assume un valore intero, si ha che il vincolo

$$\sum_{j=1}^{n} g_j x_j \leq \gamma$$

è soddisfatto da ogni $x \in F$ e quindi è una disuguaglianza valida per $P$. ♦

<u>Esempio: applicazione delle dusuguaglianze di Chvátal</u>
Consideriamo il seguente problema:

$$\begin{array}{ll} \min & -x_1 - x_2 - x_3 \\ & x_1 + x_2 \leq 1 \\ & x_2 + x_3 \leq 1 \\ & +x_1 + x_3 \leq 1 \\ & x_i \geq 0,\ x_i \in \mathbf{Z},\ i=1,\ldots,3 \end{array}$$

Questa formulazione corrisponde al problema di determinare un accoppiamento di cardinalità massima in un grafo completo di cardinalità 3. La soluzione ottima del problema è ovviamente uno e corrisponde alla scelta di un qualsiasi arco($x_1$ o $x_2$ o $x_3$ uguale a 1). Se risolviamo il problema di Programmazione Lineare che trascura i vincoli di interezza invece otteniamo 3/2 con ogni variabile fissata al valore di 1/2. Se prendiamo come moltiplicatori $u_1=u_2=u_3=1/2$, la disuguaglianza di Chvátal è:

$$\lfloor 1/2 + 1/2 \rfloor x_1 + \lfloor 1/2 + 1/2 \rfloor x_2 + \lfloor 1/2 + 1/2 \rfloor x_3 \leq \lfloor 1/2 + 1/2 + 1/2 \rfloor$$

cioè

$$x_1 + x_2 + x_3 \leq 1.$$

Aggiungendo la disuguaglianza al problema e tralasciando i vincoli di interezza otteniamo la soluzione ottima intera.

È ovvio che la generazione delle disuguaglianze valide di Chvátal può essere iterata, nel senso che si può aggiungere la disuguaglianza valida all'insieme dei vincoli e da questi produrre una nuova disuguaglianza valida e così via.
Chiameremo questa procedura **Genera_Disuguaglianza**. Il risultato molto importante stabilito da Chvátal è che è necessario un numero finito di chiamate di **Genera_Disuguaglianza** per ottenere le disuguaglianze necessarie a definire $conv(F)$. Per i dettagli di questo importante risultato si veda []

Purtroppo il risultato a cui accennavamo sopra non fornisce strumenti costruttivi, cioè non dà nessuna informazione su come determinare una disuguaglianza valida che risulti essere anche un taglio. Nei primi anni 70 Gomory ha proposto un metodo generale apparentemente infallibile per generare dei tagli per un qualsiasi problema di Programmazione Lineare Intera a partire dalla soluzione ottima frazionaria del problema in cui sono stati tralasciati i vincoli di interezza.
Si consideri il problema $P$

$$\begin{aligned} \min \quad & cx \\ & Ax = b \\ & x \geq 0 \\ & x \in \mathbf{Z}^n \end{aligned}$$

e il problema $\overline{P}$ in cui trascuriamo i vincoli di interezza

$$\begin{aligned} \min \quad & cx \\ & Ax = b \\ & x \geq 0. \end{aligned}$$

Sia $\overline{x} = \begin{bmatrix} \overline{x}_B \\ \overline{x}_N \end{bmatrix}$ una soluzione ottima di base di $\overline{P}$, quindi $\overline{x}_B, = A_B^{-1}b$ e $\overline{x}_N = 0$. Scrivendo per esteso i vincoli del problema $\overline{P}$ mettendo in evidenza le componenti in base e quelle fuori base abbiamo:

$$A_B x_B + A_N x_N = b$$

quindi ricavando per esteso l'espressione di $x_B$ abbiamo:

$$x_B = A_B^{-1}b - A_B^{-1}A_N\, x_N\,. \tag{2.4}$$

Chiamiamo $\overline{b} = A_B^{-1}b$ e $\overline{A} = A_B^{-1}A_N$. Se $\overline{x}_B$ ha tutte le componenti intere allora possiamo dire che la soluzione $\overline{x}$ è ottima anche per $P$. Altrimenti esisterà una componente $h$ tale che $\overline{x}_h$ ha un valore non intero, e sia $t$ il vincolo corrispondente alla componente $h$ nel sistema (2.4), cioè:

$$x_h + \sum_{j \in N} \overline{a}_{tj} x_j = \overline{b}_t = \overline{x}_h. \tag{2.5}$$

Osserviamo che la (2.5) è una combinazione lineare dei vincoli del problema con moltiplicatori dati dalla $t$-esima riga della matrice $A_B^{-1}$ e quindi è un vincolo valido per il problema. Consideriamo ora il troncamento dei coefficienti a sinistra, possiamo generare la seguente disuguaglianza valida:

$$x_h + \sum_{j \in N} \lfloor \overline{a}_{tj} \rfloor x_j \leq \overline{b}_t \tag{2.6}$$

Sapendo di avere a che fare con soluzioni a componenti intere possiamo affermare che la parte sinistra della (2.6) deve essere intera, quindi possiamo rafforzare la disuguaglianza nel seguente modo:

$$x_h + \sum_{j \in N} \lfloor \overline{a}_{tj} \rfloor x_j \leq \lfloor \overline{b}_t \rfloor. \tag{2.7}$$

Non ci resta che vedere che la (2.7) è un taglio, cioè che non è soddisfatta da $\overline{x}$. Dato che $\overline{x}_N = 0$, la (2.7) si riduce a:

$$\overline{x}_h \leq \lfloor \overline{b}_t \rfloor$$

che è chiaramente violata dato che $\overline{x}_h = \overline{b}_t$ .

<u>Esempio: applicazione dei tagli di Gomory (1)</u>
Si consideri il problema:

$$\begin{array}{ll} \min & -x_1 - x_2 \\ & -3x_1 + 12x_2 + x_3 = 30 \\ & 6x_1 - 3x_2 + x_4 = 8 \\ & x_i \in \mathbf{Z}_+ \end{array}$$

La soluzione ottima del problema continuo è $\bar{x}_B = \begin{bmatrix} 62/21 \\ 68/21 \end{bmatrix}$ dove $B = \{1,2\}$ e $A_B^{-1} = \begin{bmatrix} 1/21 & 4/21 \\ 2/21 & 1/21 \end{bmatrix}$.
Applicando la (2.5) alle due componenti frazionarie della soluzione abbiamo:

$$x_1 + \frac{1}{21}x_3 + \frac{4}{21}x_4 = \frac{62}{21}$$
$$x_2 + \frac{2}{21}x_3 + \frac{1}{21}x_4 = \frac{68}{21}$$

quindi i tagli di Gomory risultano:

$$x_1 \le 2, \qquad x_2 \le 3.$$

**Esercizio**
Considerando $x_3$ e $x_4$ come variabili di scarto, rappresentare graficamente il problema e i tagli di Gomory ricavati nell'esempio.

<u>Esempio: applicazione dei tagli di Gomory (2)</u>
Dato il problema di PLI

$$\begin{array}{ll} \max & -0.1x_1 + x_2 \\ & -4x_1 + 4x_2 \le 30 \\ & x_1 + 2x_2 \le 8 \\ & x_i \in \mathbf{Z}_+ \; i=1,2 \end{array}$$

La soluzione ottima può essere determinata graficamente e sta nell'intersezione dei due vincoli: $x_1$=3/2 e $x_2$=11/4. Per affrontare il problema con il metodo dei tagli di Gomory introduciamo delle variabili di scarto in modo da trasformare i vincoli in uguaglianze. Il problema così trasformato e in cui i vincoli di interezza sono rilassati è:

$$\begin{array}{ll} \max & -0.1x_1 + x_2 \\ & -4x_1 + 4x_2 + x_3 = 30 \\ & x_1 + 2x_2 + x_4 = 8 \\ & x_i \ge 0 \; i=1,,\ldots,4 \end{array}$$

La base $B$ è $\{1,2\}$ mentre $N$=$\{3,4\}$. Le varie matrici coinvolte nei calcoli sono:

$$A_B = \begin{bmatrix} -4 & 4 \\ 1 & 2 \end{bmatrix}, \qquad A_N = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A_B^{-1} = A_B^{-1}A_N = \begin{bmatrix} -1/6 & 1/3 \\ 1/12 & 1/3 \end{bmatrix}$$

Il taglio di Chvátal Gomory relativo alla prima variabile frazionaria è:

$$x_1 + \lfloor -1/6 \rfloor x_3 + \lfloor 1/3 \rfloor x_4 \le \lfloor 3/2 \rfloor$$

quindi

$$x_1 - x_3 \le 1$$

sostituendo al posto di $x_3$ l'esperssione che si deriva dalla prima equazione del sistema dei vincoli:

$$x_3 = 5 + 4x_1 - 4x_2$$

si ottiene

$$-3\,x_1 + 4\,x_2 \le 6$$

Il taglio di Chvátal Gomory relativo alla seconda variabile frazionaria è:

$$x_2 + \lfloor 1/12 \rfloor\, x_3 + \lfloor 1/3 \rfloor\, x_4 \le \lfloor 11/4 \rfloor$$

quindi

$$x_2 \le 2.$$

**Esercizio**

Rappresentare graficamente il problema, la souzione frazionaria e i tagli ottenuti. Ottenre la nuova soluzione dopo l'introduzione dei tagli.

Le tecniche illustrate sopra sono del tutto generali e si applicano a qualsiasi problema di Programmazione Lineare Intera. Esistono anche metodi che studiano più in profondità le caratteristiche dei poliedri dei vari problemi e generano disuguaglianze *ad hoc* per casi specifici. Prendiamo come esempio un problema di zaino:

$$\begin{aligned} \max \quad & \sum_{i=1}^{n} c_i\, x_i \\ & \sum_{i=1}^{n} a_i\, x_i \le b \\ & x \in \{0,1\}^n \end{aligned}$$

Se individuiamo un sottoinsieme di indici $C$ per cui

$$\sum_{i \in C} a_i > b$$

possiamo affermare che tutti gli oggetti in $C$ non possono essere presenti contemporaneamente in una soluzione ammissibile, quindi la disuguaglianza

$$\sum_{i \in C} x_i \le |C| - 1$$

è valida. In modo analogo e a volte molto sofisticato è possibile ottenere disuguaglianze valide per tanti altri problemi come ad esempio il commesso viaggiatore, il vehicle routing, la colorazione di grafi, etc.

## 3 Metodi di enumerazione implicita

I problemi di ottimizzazione discreta possono venire affrontati, in alternativa ai metodi poliedrali, per mezzo di algoritmi di tipo enumerativo. Il concetto che sta alla base di questa classe di algoritmi è abbastanza semplice: bisogna enumerare sistematicamente tutte le soluzioni ammissibili del problema, valutarne la funzione obiettivo e scegliere la migliore. A titolo di esempio consideriamo la seguente istanza del problema dello zaino:

$$\begin{aligned} \max \quad & 4x_1 + x_2 + 3x_3 + x_4 \\ & 5x_1 + 4x_2 + 3x_3 + x_4 \le 8 \\ & x_i \in \{0,1\},\ i=1,\ldots,4. \end{aligned}$$

L'insieme delle soluzioni di questo problema può essere *enumerato* utilizzando *l'albero delle decisioni* in figura 14. Consideriamo le variabili del problema (oggetti da introdurre nello zaino) una alla volta e

per ogni possibile valore attribuibile alla variabile in esame (oggetto introdotto nello zaino o scartato) "apriamo" una nuova ramificazione dell'albero delle decisioni. Ad ogni nodo dell'albero delle decisioni corrisponde quindi un sottoproblema in cui alcune variabili hanno valori prefissati determinabili in base al percorso che congiunge il nodo alla radice dell'albero. Per esempio al nodo *D* corrisponde il sottoproblema in cui abbiamo fissato $x_1 = 0$ e $x_2 = 1$:

$$
\begin{aligned}
1+\max \quad & 3x_3 + x_4 \\
& 3x_3 + x_4 \leq 8 - 4 \\
& x_i \in \{0,1\},\ i=3, 4.
\end{aligned}
$$

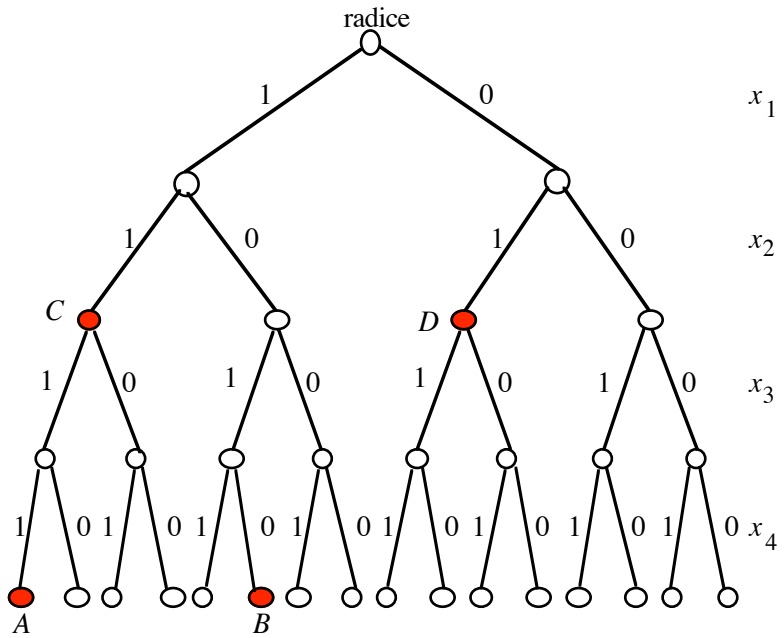


fig. 14: albero delle decisioni per un problema di zaino 0-1 con 4 oggetti

I nodi del livello *i* dell'albero sono $2^i$, dato che nel nostro caso 2 sono i valori che attribuiamo a ogni variabile. I cammini dalla radice alle foglie individuano le attribuzioni di valori {0,1} alle variabili e quindi soluzioni, non necessariamente ammissibili, per il problema dato. Ad esempio il cammino dalla radice alla foglia *A* individua una soluzione non ammissibile del problema dello zaino considerato (il peso complessivo è pari a 13 e la capacità è 8), mentre il cammino dalla radice alla foglia *B* individua una soluzione ammissibile con valore 7. Pertanto si tratta di andare ad individuare tra tutte le foglie corrispondenti a soluzioni ammissibili quella di valore migliore.

Osserviamo come i nodi dell'albero delle decisioni crescono in modo esponenziale con la dimensione del problema, pertanto un algoritmo che effettivamente procede alla enumerazione esplicita di tutti i nodi risulta poco efficiente. Bisogna quindi adottare degli "stratagemmi" che ci permettano di eliminare porzioni inutili dell'albero, riducendo il numero di nodi da valutare.

Un primo criterio ci deve permettere di riconoscere al più presto quando un sottoalbero non contiene alcuna soluzione ammissibile. Nel caso della figura 14, arrivati al nodo *C* ci accorgiamo di aver già superato la capacità dello zaino e risulta inutile procedere con l'enumerazione del sottoalbero di radice *C*. Interi sottoalberi possono così essere scartati e considerati visitati implicitamente, si parla allora di *algoritmi di enumerazione implicita*. Procedendo alla riduzione con questo criterio rimane l'albero in figura 15.

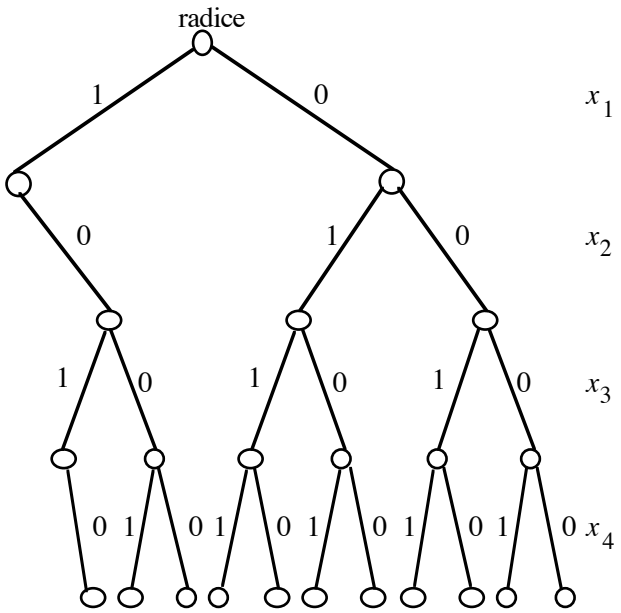


fig. 15: albero delle decisioni ridotto secondo il criterio di ammissibilità

Un secondo criterio che permette di ridurre l'ampiezza dell'enumerazione è legato al valore delle soluzioni. Supponiamo di avere a disposizione una soluzione ammissibile di valore $v$. Per esempio, nel caso dello zaino potremmo trovare una soluzione ammissibile con l'algoritmo visto nel capitolo 1 (problema del rapinatore): $x_3=x_4=1$, $x_1=0$, $x_2=1$, $v=5$. Se nello sviluppo dell'albero delle decisioni ci accorgiamo che in un sottoalbero non sono contenute soluzioni di valore maggiore di 5, è inutile procedere nell'enumerazione del sottoalbero, anche se contiene soluzioni ammissibili. Il punto più critico è come stimare il valore della migliore soluzione di un sottoalbero senza ricorrere all'enumerazione. Nel caso del nostro esempio una sovrastima molto grossolana del valore della migliore soluzione potrebbe essere quella che sceglie gli oggetti da introdurre nello zaino trascurando completamente il vincolo di capacità. Se la sovrastima calcolata in un nodo dell'albero risulta minore o uguale rispetto al valore della soluzione a disposizione allora possiamo evitare di continuare l'enumerazione del sottoalbero. Con questo criterio l'albero di enumerazione si riduce ulteriormente come illustrato in figura 16. A fianco di ogni nodo è stato riportato il valore della stima. Si noti che sulle foglie la stima coincide con il valore della soluzione.

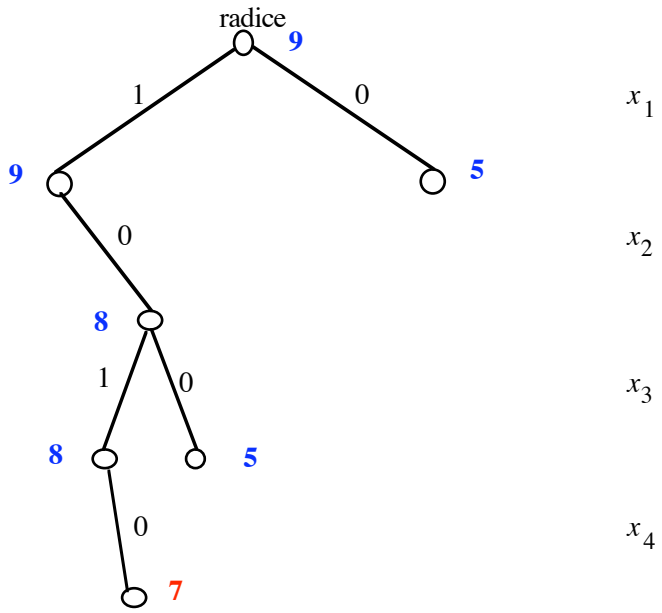


fig. 16: albero delle decisioni ridotto secondo il criterio di sovrastima della soluzione ottima

Con l'esempio sviluppato sopra abbiamo dato le idee principali di un algoritmo di enumerazione implicita, conosciuto come *Branch&Bound* che può venire riassunto dalle seguenti poche linee di codice: la procedura riceve in input un problema *P* in forma di massimizzazione e il valore di una soluzione ammissibile *v*.

```
Procedure Branch_and_Bound(P,v);
  begin
    b=Bound(P,x,infeasible);
    if x is integer then return (b);
    else if infeasible return(-∞);
      else if b>v then
      begin
        Branch(P,x,P1,P2,…,Pk)
        for i=1,…,k do begin
          t:= Branch_and_Bound(Pi,v)
          if t > v then v:= t;
        end
    end
  end.
```

fig. 17: procedura Branch&Bound

All'interno dell'algoritmo si fa uso di due procedure. La funzione **Bound**(P,x,infeasible) fornisce una stima superiore del valore della soluzione ottima del sottoproblema *P* (*upper bound*), restituendo, oltre al valore della stima, una soluzione *x* e un valore logico che vale **true** se il sottoproblema *P* non ammette soluzioni. Se infeasible=true è inutile che procediamo all'enumerazione del sottoalbero corrispondente al sottoproblema *P* poiché non contiene soluzioni. Al contrario se **Bound**(P,x,infeasible) resituisce una soluzione *x* a componenti intere, quindi ammissibile è altrettanto inutile procedere all'enumerazione del sottoalbero corrispondente al sottoproblema *P*, poiché disponiamo già del valore della sua soluzione ottima. L'altra procedura utilizzata è quella che decompone il problema *P* in sottoproblemi $P_1, \ldots, P_k$. effettuando l'operazione di "ramificazione" dell'albero di enumerazione. La procedura **Branch** riceve in input la soluzione *x* fornita dal Bound dato che in genere fornisce indicazioni preziose su come partizionare efficientemente *P*.

Gli ingredienti fondamentali di un algoritmo di Branch&Bound sono ora evidenti: 1) la tecnica di stima del "bound" che deve essere efficiente in termini di calcolo e quanto più accurata possibile per ridurre l'ampiezza dell'albero, 2) la decomposizione del problema, 3) il criterio di visita dell'albero di enumerazione.

### *3.1 Stima della soluzione ottima: rilassamenti*

Consideriamo un problema di ottimizzazione $P$ : $\max\{c(x): x \in F\}$.

**Definizione 3.1**
Chiamiamo $P'$: $\max\{c'(x): x \in F'\}$ un *rilassamento* di $P$ se

$$F \subseteq F',$$
$$x \in F \Rightarrow c'(x) \geq c(x).$$

In base alla definizione di rilassamento è facile verificare il seguenti risultati:

**Teorema 3.2**
Il valore del rilassamento di un problema di massimizzazione costituisce una stima per eccesso del valore della soluzione ottima:

$$z(P) = \max\{c(x): x \in F\} \leq z(P') = \min\{c'(x): x \in F'\}.$$

**Teorema 3.3**
Sia $x^* \in F'$ una soluzione ottima per il rilassamento $P'$, cioè $z(P') = c'(x^*)$, e sia inoltre $x^* \in F$ e $c(x^*)=c'(x^*)$, allora $x^*$ è anche soluzione ottima per $P$

Nel definire un rilassamento di un problema generalmente si tiene conto del fatto che la soluzione ottima di un rilassamento dovrebbe essere più semplice da calcolare rispetto a quella del problema originale. Nel seguito considereremo solamente alcune fra le principali tecniche di rilassamento e cioè la *eliminazione di vincoli* e il *rilassamento continuo*.

Si consideri un problema $P$ la cui regione ammissibile è data dall'intersezione di due insiemi di vincoli:

$$(P) \qquad \min\{f(x) : x \in X_1 \cap X_2\}.$$

Un rilassamento di $P$ è il problema

$$(PR) \qquad \min\{f(x) : x \in X_1\},$$

ottenuto da $P$ eliminando il vincolo che $x$ debba appartenere all'insieme $X_2$.
Un rilassamento di questo tipo ha senso quando i vincoli che definiscono l'insieme $X_1 \cap X_2$ sono difficili da trattare, mentre sono trattabili quelli che definiscono solamente l'insieme $X_1$.

<u>Esempio: rilassamento del problema del commesso viaggiatore</u>
Consideriamo il problema del commesso viaggiatore su grafo orientato discusso negli esempi del paragrafo 1. Un rilassamento comune alle due formulazioni TSP1 e TSP2 è il problema di assegnamento:

$$\text{TSPR:} \ \min\{\sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij} : x \in X_1\}.$$

Le soluzioni di questo rilassamento possono essere oltre che cicli Hamiltoniani anche insiemi di cicli disgiunti contenenti tutti i nodi del grafo (copertura per cicli dei nodi del grafo).

Il rilassamento continuo, volendo, è un caso particolare di eliminazione di vincoli. Infatti nel rilassamento continuo di un problema di Programmazione Lineare Intera si eliminano i vincoli di interezza delle variabili. Sia dato il problema di ottimizzazione combinatoria:

$$(P) \qquad \min\{f(x): x \in X, x \in \{0,1\}^n\},$$

con $X$ sottoinsieme convesso di $\boldsymbol{R}^n$.
Si dice *rilassamento continuo* di $P$ il seguente problema

$$(Pc) \qquad \min\{f(x): x \in X, x \in [0,1]^n\}.$$

Se $P$ è un problema di programmazione intera

$$(P) \qquad \min\{f(x): x \in X,\ x \in \boldsymbol{Z}^n\},$$

allora il rilassamento continuo di $P$ è dato da:

$$(Pc) \qquad \min\{f(x): x \in X\}.$$

In entrambi i casi, se $f(x)$ è lineare e $X$ è un poliedro, $Pc$ è un problema di programmazione lineare.

<u>Esempio: localizzazione di impianti rilassamento continuo</u>
Prendiamo in esame un problema di localizzazione di impianti simile a quello discusso nel capitolo 1. È dato un insieme di siti candidati $S$ in cui è possibile installare dei centri di servizio (nel capitolo 1 si trattava di antenne cellulari). Per ogni sito candidato $j \in S$ è noto un costo di attivazione $f_j > 0$ e una capacità massima $u_j$. È dato anche un insieme di clienti $C$ da assegnare ai centri attivati. Per ogni possibile assegnamento cliente $i$ - centro $j$ è noto un coefficiente di utilità $c_{ij} > 0$. Il problema consiste nell'attivare un sottoinsieme di centri e assegnare loro i clienti in modo da rispettare i vincoli di capacità e minimizzare la differenza tra costo e utilità. Una possibile formulazione utilizza variabili 0-1 di attivazione $y_j$, e di assegnamento $x_{ij}$:

$$\begin{array}{lll}
\min & \sum_{j \in S} f_j y_j - \sum_{i \in C} \sum_{j \in S} c_{ij} x_{ij} & \\
& \sum_{j \in S} x_{ij} = 1 & \forall i \in C \\
& \sum_{i \in C} x_{ij} \le u_j y_j & \forall j \in S \\
& x_{ij}, y_j \in \{0,1\} & \forall i \in C, \forall j \in S.
\end{array}$$

Il rilassamento continuo del problema di localizzazione si ottiene sostituendo i vincoli di appartenenza all'insieme $\{0,1\}$ delle variabili $x$ e $y$, con i vincoli:

$$0 \le x_{ij} \le 1,\ 0 \le y_j \le 1 \qquad \forall i \in C, \forall j \in S.$$

Chiaramente, ne risulta un problema di programmazione lineare, che possiamo risolvere efficientemente con l'algoritmo del simplesso. Possiamo però spingere oltre la nostra analisi e constatare come in realtà il rilassamento continuo possa venire ricondotto a un problema di flusso di costo minimo. Il grafo è definito da un insieme di nodi $N$ corrispondenti ai clienti in $C$, ai centri in $S$ e a un nodo *pozzo* $t$. In ogni nodo-cliente vi è una offerta di una unità di flusso, il pozzo $t$ richiede $|C|$ unità di flusso, mentre i nodi-centro sono di transito. Abbiamo archi che collegano tra loro tutti i nodi-

cliente a tutti i nodi centro con capacità 1, e tutti gli archi che collegano ogni nodo-centro $j$ a $t$ con capacità $u_j$. I costi degli archi sono pari a $-c_{ij}$ per quelli dal cliente $i$ al centro $j$, mentre sono $f_j/u_j$ per quelli dal centro $j$ a $t$. In figura 18 abbiamo riportato una rappresentazione della rete.

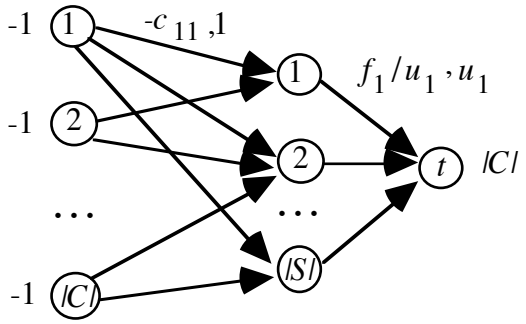


fig. 18: formulazione del rilassamento continuo del problema di localizzazione

Il problema può venire risolto molto efficientemente con un algoritmo per il flusso di costo minimo. Le variabili $x_{ij}$ corrispondono ai flussi sugli archi $(i,j)$ mentre le $y_j$ vengono ottenute dividendo il flusso sull'arco $(j,t)$ per $u_j$.

***Esercizio**

Per il problema di localizzazione possiamo ricorrere ad un altro rilassamento che dà luogo ad una serie di problemi noti. Consideriamo l'eliminazione dei vincoli di assegnamento ( $\sum_{j \in S} x_{ij} = 1$, $\forall i \in C$). Analizzare i problemi che ne derivano e fornire un algoritmo di soluzione.

Osserviamo come diversi modelli tra loro equivalenti possano produrre rilassamenti continui diversi (non tutti ugualmente efficaci).

Ad esempio si consideri il seguente vincolo logico:

$$y = 0 \;\Rightarrow\; x_1 = 0 \text{ e } x_2 = 0, \text{ con } y, x_1 \text{ e } x_2 \text{ variabili binarie.}$$

Tale vincolo può essere espresso indifferentemente per mezzo di uno dei seguenti modelli:

$$M_1: \quad x_1 + x_2 \le 2y, \quad (x_1, x_2, y) \in \{0,1\}^3;$$

$$M_2: \quad x_1 \le y\,,\; x_2 \le y, \quad (x_1, x_2, y) \in \{0,1\}^3.$$

I rilassamenti continui sono rispettivamente:

$$M'_1: \quad x_1 + x_2 \le 2y, \quad (x_1, x_2, y) \in [0,1]^3;$$

$$M'_2: \quad x_1 \le y\,,\; x_2 \le y, \quad (x_1, x_2, y) \in [0,1]^3.$$

È facile verificare che $M'_1 \supset M'_2$; infatti mentre chiaramente ogni punto di $M'_2$ appartiene anche a $M'_1$, il contrario non è vero. Ad esempio i punti $(1, 0, \frac{1}{2})$ e $(0, 1, \frac{1}{2})$ appartengono a $M'_1$ ma non a $M'_2$.

Quindi è in generale preferibile il secondo rilassamento al primo, anche se richiede un vincolo in più, laddove si voglia avere dei rilassamenti il più possibile *vicini* al problema originale.

<u>Esempio: calcolo efficiente del rilassamento continuo per il problema dello zaino</u>

Consideriamo un problema di zaino con $n$ oggetti:

$$
\begin{aligned}
(P)\colon \max \sum_{i=1}^{n} c_i x_i & \\
\sum_{i=1}^{n} a_i x_i \le b & \\
x_i \in \{0,1\} \qquad & i=1,\ldots,n.
\end{aligned}
$$

Il rilassamento continuo $\overline{P}$ di $P$ si ottiene trasformando i vincoli $x_i \in \{0,1\}, i=1,\ldots,n$ in $0 \le x_i \le 1$, $i=1,\ldots,n$.

$$
\begin{aligned}
(\overline{P})\colon \max \sum_{i=1}^{n} c_i x_i & \\
\sum_{i=1}^{n} a_i x_i \le b & \\
0 \le x_i \le 1, \qquad & i=1,\ldots,n.
\end{aligned}
$$

Supponendo per semplicità che gli indici degli oggetti siano già ordinati in modo non crescente di rapporto $(c_i/a_i)$, la soluzione ottima di $\overline{P}$ può venire calcolata secondo la seguente regola: sia $k$ l'indice tale che:

$$
\sum_{i=1}^{k-1} a_i < b
$$

$$
\sum_{i=1}^{k} a_i \ge b
$$

allora

$$
\overline{x}_i = \begin{cases} 1, & i=1,\ldots,k-1 \\ \dfrac{b - \sum_{j=1}^{k-1} a_j}{a_k}, & i=k, \\ 0 & i=k+1,\ldots,n. \end{cases}
$$

Si noti che la soluzione così calcolata ha al massimo una variabile con valore frazionario $\overline{x}_k$, mentre tutte le altre valgono 0 o 1. Possiamo dimostrare che la soluzione così determinata è effettivamente ottima ricorrendo al teorema degli scarti complementari. Il duale del problema $(\overline{P})$ è:

$$
\begin{aligned}
(\overline{D})\colon \min \lambda b + \sum_{i=1}^{n} \mu_i & \\
\lambda a_i + \mu_i \ge c_i \qquad & i=1,\ldots,n \\
\lambda, \mu_i \ge 0, \qquad & i=1,\ldots,n.
\end{aligned}
$$

dove la variabile $\lambda$ è associata al vincolo di capacità in $(\overline{P})$ e le variabili $\mu_i$ sono associate ai vincoli $(x_i \le 1)$. Le equazioni degli scarti complementari sono:

$$(\lambda a_i + \mu_i - c_i)\, x_i = 0 \qquad i=1,\ldots,n$$
$$\lambda\, (b - \sum_{i=1}^{n} a_i x_i\,) = 0$$
$$\mu_i\, (1 - x_i\,) = 0 \qquad i=1,\ldots,n.$$

Da come è stata costruita la soluzione $\bar{x}$, applicando le equazioni degli scarti complementari otteniamo la seguente soluzione duale:

$$\bar{\mu}_i = 0 \qquad i = k,\ldots,n$$
$$\bar{\lambda} = \frac{c_k}{a_k}$$
$$\bar{\mu}_i = c_i - \frac{c_k}{a_k} a_i \qquad i = 1,\ldots,k\text{-}1.$$

**Esercizio**

Dimostrare che la soluzione $(\bar{\lambda},\bar{\mu})$ è duale ammissibile.

Proviamo ad applicare la costruzione della soluzione ottima del rilassamento continuo al seguente problema:

$$\begin{aligned} P\text{: max } & 52x_1 + 40x_2 + 38x_3 + 9x_4 + x_5 \\ & 17x_1 + 5x_2 + 13x_3 + 3x_4 + x_5 \leq 20 \\ & x_i \in \{0,1\},\ i=1,\ldots,5. \end{aligned}$$

L'ordinamento degli indici secondo il rapporto $c_i/a_i$ è" 2, 1, 4, 3, 5. La soluzione del rilassamento continuo è quindi: $\bar{x}_2 = 1$, $\bar{x}_1 = 15/17$, $\bar{x}_4 = \bar{x}_3 = \bar{x}_5 = 0$, di valore 85.88.

### *3.2 Regole di branching*

In un algoritmo di enumerazione la *regola di branching* permette di partizionare la regione ammissibile in sottoinsiemi, ognuno dei quali definisce un sottoproblema di dimensioni ridotte rispetto al problema originale. Formalmente, se stiamo affrontando un problema del tipo

$$P\text{: max } \{cx\text{: } x \in F\}$$

una regola di branching definisce una partizione di $F$ in una famiglia di sottoinsiemi $\{F_1, F_2, \ldots, F_k\}$, in cui ovviamente l'unione dei sottoinsiemi fornisce l'insieme originale $F$ e possibilmente la loro intersezione a due a due è vuota. I sottoproblemi su cui continuiamo l'esplorazione sono:

$$P_i\text{: max } \{cx\text{: } x \in F_i\} \qquad i=1, \ldots, k.$$

Quando $k$=2 parliamo di regola di branching bipartita.

Un'altra caratterizzazione di una regola di branching è la strategia che guida la scelta della partizione. Nell'esempio che illustrava i principi di base degli algoritmi di enumerazione implicita avevamo adottato una regola di branching bipartita. In quell'esempio avevamo fissato un ordinamento delle variabili che venivano considerate in sequenza nei vari livelli dell'albero di enumerazione. Ad ogni nodo dell'albero, la variabile relativa al livello veniva fissata (a 0 e a 1) e l'albero subiva una ramificazione. Questa strategia di enumerazione estremamente statica può venire migliorata tenendo conto delle caratteristiche del problema da risolvere e della soluzione del rilassamento continuo. Innanzitutto non è necessario che in nodi dello stesso livello dell'albero venga necessariamente effettuato il branching sulla medesima variabile.

La scelta della variabile su cui effettuare il branching è solitamente legata al risultato del rilassamento del problema. Consideriamo un problema di Programmazione Lineare Intera con variabili binarie

$$\begin{aligned} \max \quad & cx \\ & Ax \leq b \\ & x \in \{0,1\}^n \end{aligned}$$

e sia $\bar{x}$ la soluzione ottima del suo rilassamento continuo che supponiamo non essere ammissibile per il problema originale. Dovendo ramificare l'albero di enumerazione siamo liberi di scegliere qualsiasi variabile. Se scegliessimo una variabile $x_i$ per cui $\bar{x}_i$ è uguale a 0, avremmo due sottoproblemi $P_0 = \{\max cx\colon Ax \leq b, x \in \{0,1\}^n, x_i = 0\}$ e $P_1 = \{\max cx\colon Ax \leq b, x \in \{0,1\}^n, x_i = 1\}$. Si noti che la soluzione ottima del rilassamento continuo di $P_0$ è uguale a $\bar{x}$, poiché aver fissato la variabile $x_i = 0$ non ha influenzato la soluzione già ottenuta. Analogo ragionamento vale per il problema $P_1$ quando fissiamo a 1 una variabile per cui $o(x, \bar{\ })_i$ è uguale a 1. In questi casi avremmo quindi modo di evitare il calcolo del rilassamento continuo in uno dei due rami dell'albero, ma rischiamo anche di "catturare" poca informazione dalla soluzione del rilassamento continuo. L'idea utilizzata generalmente è quella di scegliere una variabile $x_i$ per cui $\bar{x}_i$ assume valori frazionari. Questo implicherebbe che in entrambi i rami dell'albero di enumerazione avremmo un cambiamento del rilassamento continuo. Quando vi è la possibilità di scegliere tra più variabili frazionarie, generalmente si privilegia quella "più frazionaria", ovvero, nel caso di problemi a variabili binarie, quella il cui valore nel rilassamento continuo è più vicino a 0.5.

<u>Esempio: branching per il problema di zaino binario</u>

Consideriamo il problema di zaino binario di cui abbiamo calcolato il rilassamento continuo in precedenza. La variabile su cui effettuare il branching è l'unica con valore frazionario quindi $x_1$. Otteniamo i due sottoproblemi:

$$\begin{aligned} P_0\colon \max \quad & 40x_2 + 38x_3 + 9x_4 + x_5 \\ & 5x_2 + 13x_3 + 3x_4 + x_5 \leq 20 \\ & x_i \in \{0,1\}, \qquad i=2,\ldots,5. \end{aligned}$$

$$\begin{aligned} P_1\colon 52 + \max \quad & 40x_2 + 38x_3 + 9x_4 + x_5 \\ & 5x_2 + 13x_3 + 3x_4 + x_5 \leq 3 \\ & x_i \in \{0,1\}, \qquad i=2,\ldots,5. \end{aligned}$$

I cui rilassamenti continui sono rispettivamente: $\bar{x}_2 = 1$, $\bar{x}_4 = 1$, $\bar{x}_3 = 12/13$, $\bar{x}_5 = 0$, di valore 84.08 e $\bar{x}_2 = 3/5$, $\bar{x}_4 = \bar{x}_3 = \bar{x}_5 = 0$ di valore 76. Si noti che volendo continuare ad effettuare il branching, mentre in $P_0$ la variabile di riferimento è $x_3$, in $P_1$ è $x_2$, e dà luogo all'albero rappresentato in figura 19.

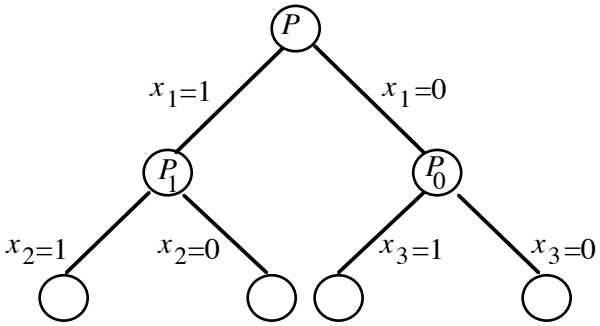


fig. 19: branching fino al secondo livello dell'albero di enumerazione

Quando siamo nel caso più generale di problemi a variabili intere,

$$
\begin{aligned}
P\colon \max\ & cx \\
& Ax \le b \\
& x \in \mathbf{Z}^n
\end{aligned}
$$

possiamo continuare ad adottare la regola di branching binario. Consideriamo una variabile $x_i$ per cui la soluzione del rilassamento continuo $\bar{x}_i$ assume un valore frazionario. La decomposizione in due sottoproblemi può essere la seguente:

$$
\begin{aligned}
P_L &= \{\max cx\colon Ax \le b,\ x \in \mathbf{Z}^n,\ x_i \le \lfloor \bar{x}_i \rfloor\} \\
P_U &= \{\max cx\colon Ax \le b,\ x \in \mathbf{Z}^n,\ x_i \ge \lceil \bar{x}_i \rceil\}.
\end{aligned}
$$

Si noti che in questo caso non sono state fissate delle variabili ma solo stati introdotti dei vincoli. L'introduzione dei vincoli ha permesso di eliminare tutti i valori della variabile $x_i$ compresi tra $\lfloor \bar{x}_i \rfloor$ e $\lceil \bar{x}_i \rceil$.

<u>Esempio: branching bipartito per il caso generale di PLI</u>
Prendiamo il seguente problema di programmazione lineare intera:

$$
\begin{aligned}
P\colon \max\ & -x_1 + x_2 \\
& x_1 + x_2 \le 5 \\
& -3x_1 + 2x_2 \le 1 \\
& 2x_1 \le 7 \\
& x_i \in \mathbf{Z}_+,\ i=1, 2.
\end{aligned}
$$

In figura 20 abbiamo la rappresentazione geometrica di $P$ e la soluzione del suo rilassamento lineare che è data da $\bar{x}_1 = 9/5$, $\bar{x}_2 = 16/5$.

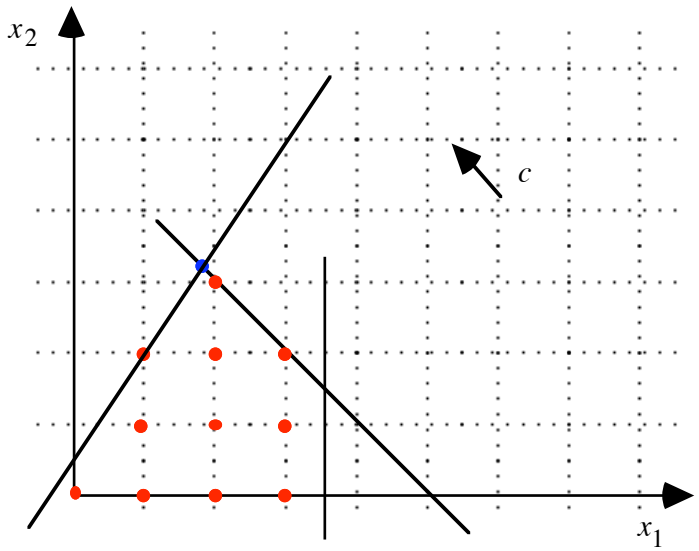


fig. 20: rappresentazione geometrica e soluzione del rilassamento continuo

Scegliendo di effettuare la variabile $x_1$ Otteniamo i due sottoproblemi:

$$
\begin{array}{lll}
P_L\text{: max } -x_1 + x_2 & & \\
\quad x_1 + x_2 & \leq & 5 \\
\quad -3x_1 + 2x_2 & \leq & 1 \\
\quad 2x_1 & \leq & 1 \\
\quad x_i \in \mathbf{Z}_+,\ i=1,2. & &
\end{array}
$$

$$
\begin{array}{lll}
P_U\text{: max } -x_1 + x_2 & & \\
\quad x_1 + x_2 & \leq & 5 \\
\quad -3x_1 + 2x_2 & \leq & 1 \\
\quad 2x_1 & \leq & 7 \\
\quad 2x_1 & \geq & 2 \\
\quad x_i \in \mathbf{Z}_+,\ i=1,2. & &
\end{array}
$$

In figura 21 sono evidenziate le regioni ammissibili dei due sottoproblemi.

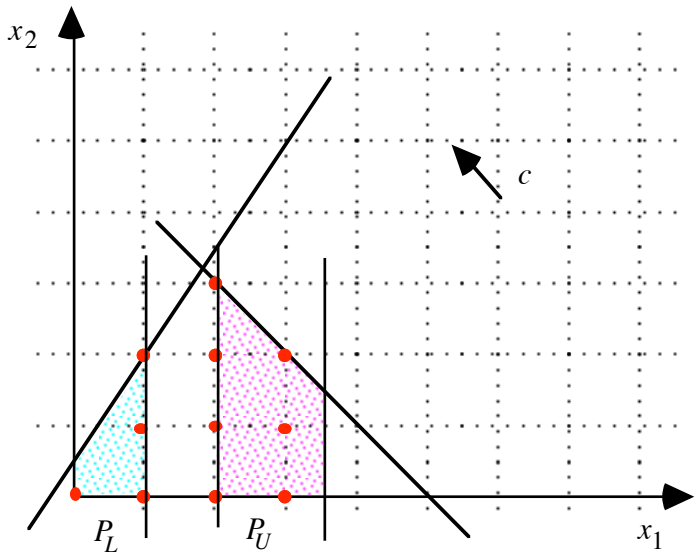


fig. 21: rappresentazione geometrica e soluzione del rilassamento continuo

**Esercizio**
Sviluppare l'albero di enumerazione per il problema $P$ valutando la soluzione dei rilassamenti continui ricorrendo alla rappresentazione geometrica di figura 21.

Sviluppando un albero di enumerazione per alcuni problemi potrebbe risultare interessante non rimanere legati a una regola di branching bipartito. Per esempio in un problema di PLI del tipo

$$
\begin{array}{ll}
P\text{: max} & cx \\
& Ax \leq b \\
& x \in \{1,\ldots,k\}^n
\end{array}
$$

cioè in cui le variabili assumono valori interi in un insieme limitato, una volta individuata la variabile su cui effettuare il branching ($x_i$ ) potrebbe essere utile effettuare la seguente decomposizione:

$$
\begin{array}{l}
P_1 = \{\max cx\text{: } Ax \leq b,\ x \in \{1,\ldots,k\}^n,\ x_i = 1\} \\
\ldots \\
P_k = \{\max cx\text{: } Ax \leq b,\ x \in \{1,\ldots,k\}^n,\ x_i = k\}\ ,
\end{array}
$$

quindi fissando il valore della variabile di branching a turno a tutti i valori possibili nel suo insieme di ammissibilità. Parliamo in questo caso di *branching k-partito*.
Una regola di branching $k$-partita può venire adottata anche in alcuni casi di problemi a variabili binarie come vediamo dal seguente esempio.

Esempio: branching per il problema del commesso viaggiatore
Sia $G=(N,A)$ un grafo non orientato completo con pesi $w_{ij}\geq 0$ associati agli archi, e si voglia determinare il ciclo Hamiltoniano di peso complessivo minimo. Nel capitolo due abbiamo visto come questo problema possa venire formulato utilizzando variabili decisionali binarie associate agli archi. Quindi verrebbe naturale impostare un albero di enumerazione basandosi su una regola di branching binaria. Si noti però che ne variabili sono $\frac{n(n-1)}{2}$ quindi le foglie dell'albero di enumerazione completo saranno $2^{n(n-1)/2}$, mentre il numero di soluzioni ammissibili, ovvero il numero di cicli Hamiltoniani di un grafo completo è $(n-1)!$, cioè un numero molto inferiore rispetto alle foglie dell'albero. Questo significa che moltissime foglie corrisponderanno a soluzioni non ammissibili e potrebbe essere gravoso dover tener conto della loro eliminazione.
Una regola di branching alternativa può essere di tipo $k$-partito. Consideriamo il nodo radice dell'albero di enumerazione e supponiamo di trovarci nel nodo 1 del grafo dovendo iniziare la visita. Le alternative che ci si pongono sono $(n-1)$, tante quante sono i nodi non ancora visitati. Possiamo allora aprire $(n-1)$ rami dell'albero, uno per ogni nodo successivo al primo. Nei livelli successivi dell'albero le scelte si riducono (una in meno per ogni livello). fino ad arrivare al livello $(n-2)$ dove ovviamente la scelta è obbligata. Vi è una corrispondenza 1:1 tra le foglie dell'albero e le permutazioni di $n$ nodi che iniziano con il nodo 1, quindi con le soluzioni ammissibili del problema del commesso viaggiatore.

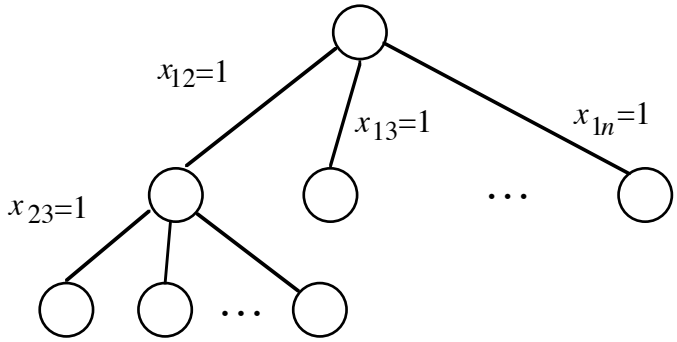


fig. 22: una regola di branching $k$-partita per il problema del commesso viaggiatore

Si noti che quando scegliamo il nodo successivo nel ciclo rispetto al nodo $i$, sia esso $j$, in pratica fissiamo la variabile $x_{ij}$ a 1. Contemporaneamente a questa scelta fissiamo implicitamente tutte le variabili $x_{ih}$ a 0 non potendo avere altri nodi successivi a $i$ oltre quello scelto.
La strategia di branching che abbiamo proposto è statica e non è in alcun modo legata alle informazioni che possono pervenire dal calcolo del rilassamento del problema. Proponiamo ora una seconda regola di branching $k$ partito basata sulle informazioni che provengono dal rilassamento in cui si trascurano i vincoli di eliminazione dei sottocicli (*TSPR*). Se la soluzione del rilassamento non è ammissibile, significa che esiste un ciclo che non contiene tutti i nodi del grafo. Per semplicità supponiamo che la soluzione del rilassamento induca un ciclo di tre nodi $\{i - j - h\}$, Una regola di branching che tiene conto di questa informazione partiziona l'insieme delle soluzioni cercando di "spezzare" tale ciclo, e quindi di impedire che gli archi che compongono il sottociclo siano presenti contemporaneamente nella soluzione. I sottoproblemi che vengono generati sono:

$P_1$: {min $cx$: $x \in X$, $x_{ij}=0$}
$P_2$: {min $cx$: $x \in X$, $x_{ij}=1$, $x_{jh}=0$,}
$P_3$: {min $cx$: $x \in X$, $x_{ij}=1$, $x_{jh}=1$, $x_{hi}=0$}.

**Esercizio**
Dimostrare che la regola di branching proposta per il problema del commesso viaggiatore partiziona correttamente la regione ammissibile senza escludere nessun ciclo Hamiltoniano.

### *3.3 Strategie di visita dell'albero di enumerazione*

Un ulteriore elemento che caratterizza un algoritmo di Branch and Bound è il criterio con il quale si sceglie il prossimo nodo dell'albero di enumerazione da esplorare. In genere vengono distinti due criteri fondamentali: *best first* e *depth first*.
La semplice procedura di Branch and Bound riassunta in figura 17, per la sua natura ricorsiva, fa la scelta di esplorare subito in sequenza i nodi generati dall'operazione di branching. In un nodo dell'albero di enumerazione durante l'esecuzione dell'algoritmo si possono verificare due casi. Nel primo caso il bound del nodo è maggiore del valore della migliore soluzione trovata (nel caso di un problema di massimizzazione), e quindi è richiesta una ulteriore esplorazione. Questo significa che il sottoproblema viene decomposto e la procedura passa ad esplorare un nodo di profondità maggiore. Nel secondo caso il bound fornisce una soluzione ammissibile, sancisce la inammissibilità del sottoproblema o verifica che le eventuali soluzioni ammissibili per il sottoproblema non sono interessanti per risolvere il problema generale. In tal caso la procedura ricorsiva termina senza ulteriormente decomporre il problema e l'algoritmo passa ad esplorare un altro nodo di livello superiore, effettuando una cosiddetta operazione di *backtracking*. È evidente che questa implementazione dell'algoritmo effettua una visita in profondità dell'albero (depth first) privilegiando il raggiungimento rapido di una foglia dell'albero, e quindi la generazione rapida di una soluzione ammissibile anche se nulla garantisce che la soluzione generata sia di buona qualità. Un ulteriore vantaggio di questo tipo di strategia di visita dell'albero di enumerazione è dato dal fatto che i requisiti di memoria dell'esecuzione dell'algoritmo sono minimi. Infatti durante l'esecuzione avremo al più $k$ nodi "aperti" per ogni livello dell'albero, dove $k$ è il massimo numero di nodi generati dalla regola di branching. Il principale inconveniente è invece dato dal fatto che, se non si dispone di una buona soluzione iniziale, l'eliminazione di sottoproblemi dall'albero di enumerazione grazie alla regola del bound diventa più improbabile, quindi il numero di nodi da enumerare può essere molto elevato.
Per cercare di indirizzare l'esplorazione dell'albero di enumerazione verso la generazione di soluzioni di buona qualità, e quindi rendere più efficace la regola di eliminazione di sottoproblemi, potremmo scegliere di esplorare prima i nodi che hanno fornito un bound migliore (da cui il nome di criterio "best first"), quindi più elevato nel caso di problemi di massimizzazione. Questo tipo di implementazione dell'algoritmo non può essere fatto in una semplice forma ricorsiva, ma richiede l'introduzione di un'opportuna struttura dati che memorizza i nodi ancora da esplorare e dalla quale viene estratto ad ogni iterazione il nodo con valore del bound massimo. Questo criterio porta ad esplorare prima le zone dell'albero più "interessanti" e che contengono la soluzione ottima con maggiore probabilità. Purtroppo però, per le caratteristiche dei rilassamenti, i bound migliori vengono forniti dai nodi di livello alto nell'albero di enumerazione. Questo implica che il numero di nodi "aperti" può essere molto elevato (esponenziale nel numero di livelli dell'albero, in linea di principio) e i requisiti di memoria dell'esecuzione dell'algoritmo sono gravosi.
In letteratura esistono anche molti algoritmi di tipo "ibrido" che combinano i due criteri di visita dell'albero. Per esempio, ponendo un limite massimo al numero di nodi contemporaneamente aperti, si

può procedere con una visita best first fino a che tale limite non viene raggiunto, per poi continuare con il criterio depth first che non incide sull'impiego di memoria da parte dell'algoritmo.

**Esercizio**
Descrivere formalmente lo pseudocodice dell'algoritmo di Branch and Bound con il criterio best first. Quale struttura dati si può utilizzare per memorizzare i nodi dell'albero, tenendo conto delle operazioni di estrazione da effettuare?

## *4 Cenni di complessità computazionale

Fino ad ora abbiamo avuto modo di passare in rassegna a vari problemi di ottimizzazione e ne abbiamo studiato algoritmi per la loro soluzione. La nostra attenzione è stata rivolta principalmente agli aspetti legati alla correttezza degli algoritmi, ovvero alla garanzia di ottimalità delle soluzioni trovate. Un altro aspetto fondamentale dello studio algoritmico è quello che si lega all'efficienza degli algoritmi proposti, ovvero a una valutazione delle loro prestazioni, tipicamente in termini di tempo di esecuzione o spazio di memoria.
Per poter impostare con rigore le considerazioni legate all'efficienza degli algoritmi abbiamo necessità di introdurre un minimo di definizioni. Nel seguito intenderemo per *problema* una domanda espressa in termini generali, la cui risposta dipende da un certo numero di *parametri* e *variabili*. Un problema viene usualmente definito per mezzo di:

- una descrizione dei suoi parametri, in generale lasciati indeterminati;
- una descrizione delle proprietà che devono caratterizzare la risposta o *soluzione* che si desidera.

Ad esempio, il problema del commesso viaggiatore è specificato nel seguente modo: "Su un grafo non orientato completo, con pesi associati agli archi, trovare il ciclo Hamiltoniano di peso complessivo minimo", I parametri del problema sono il numero di nodi del grafo (non importa specificare altro visto che il grafo è completo), e ma matrice dei pesi degli archi. La caratterizzazione delle soluzioni è data dalla specifica dei Hamiltoniani, e la soluzione cercata è quella la cui somma degli archi che compongono il ciclo è minima.

Una *istanza* di un dato problema $P$ è quella particolare domanda che si ottiene specificando particolari valori per tutti i parametri di $P$.
Quindi, nel caso del commesso viaggiatore, una istanza è data specificando il valore del numero di nodi $n$ e fornendo in input $n(n-1)/2$ pesi da associare agli archi del grafo.

Formalmente, un *algoritmo* che risolve un dato problema $P$ può essere definito come una sequenza finita di istruzioni, che applicata ad una qualsiasi istanza $p$ di $P$, si arresta dopo un numero finito di passi (ovvero di computazioni elementari), fornendo una soluzione dell'istanza $p$, oppure indicando che l'istanza $p$ non ha soluzioni ammissibili.

### *4.1 Modelli computazionali e misure di complessità*

In precedenza, in particolare nel capitolo 2, abbiamo misurato abbastanza informalmente l'efficienza di un algoritmo, però per potere studiare gli algoritmi dal punto di vista della loro efficienza, o *complessità computazionale*, è necessario definire un modello computazionale.
Classici modelli computazionali sono la Macchina di Touring, storicamente il primo proposto, la R.A.M. (Random Access Machine), la Macchina a puntatori, etc.
Un buon compromesso tra semplicità e versatilità è la R.A.M.
Una R.A.M. consiste di un programma finito, di un insieme finito di registri ciascuno dei quali può contenere un singolo numero intero (o razionale), e di una memoria di $n$ parole ciascuna delle quali ha un unico indirizzo (tra 1 ed $n$) e può contenere un singolo numero intero (o razionale).

In un passo una R.A.M. può:
- effettuare una singola operazione (aritmetica o logica) sul contenuto di uno specificato registro,
- scrivere in uno specificato registro il contenuto di una parola il cui indirizzo è in un registro,
- immagazzinare il contenuto di uno specificato registro in una parola il cui indirizzo è in un registro.

Si tratta di una macchina *sequenziale* e *deterministica* (il comportamento futuro della macchina è univocamente determinato dalla sua configurazione presente).

Dato un problema $P$, una sua istanza $p$, e un algoritmo $A$ che risolve $P$, indichiamo con *costo computazionale* (o *complessit*à) di $A$ applicato a $p$ una misura delle risorse utilizzate dalle computazioni che $A$ esegue su una macchina R.A.M. per determinare la soluzione di $p$. Le risorse, in principio, sono di due tipi: *memoria occupata* e *tempo di calcolo*. Nell'ipotesi che tutte le operazioni elementari abbiano la stessa durata, il tempo di calcolo può essere espresso come numero di operazioni elementari effettuate dall'algoritmo. Poiché molto spesso la risorsa più critica è il tempo di calcolo, nel seguito useremo soprattutto il tempo di calcolo come misura della complessità degli algoritmi.

Dato un algoritmo è opportuno disporre di una misura di complessità che consenta una valutazione sintetica della bontà dell'algoritmo stesso ed eventualmente un suo agevole confronto con algoritmi alternativi. Conoscere la complessità di $A$ per ognuna delle istanze di $P$ non è possibile (l'insieme delle istanze di un problema è in generale non limitato), né sarebbe di utilità pratica. Si cerca allora di esprimere la complessità come una funzione $g(n)$ della dimensione, $n$, dell'istanza cui viene applicato l'algoritmo; ma poiché, per ogni dimensione, si hanno in generale molte istanze di quella dimensione, si sceglie $g(n)$ come il costo necessario per risolvere la più difficile tra le istanze di dimensione $n$. Si parla allora di *complessità nel caso peggiore*.
Bisogna naturalmente definire in modo preciso il significato di dimensione di una istanza. Chiameremo *dimensione* di una istanza $p$ una misura del numero di bit necessari per rappresentare i dati che definiscono $p$ con una codifica "ragionevolmente" compatta, cioè una misura della lunghezza del suo input. Per esempio in un grafo con $n$ nodi e $m$ archi, i nodi possono essere rappresentati per mezzo degli interi tra 1 ed $n$, mentre gli archi per mezzo di una lista contenente $m$ coppie di interi (l'arco che collega i nodi $i$ e $j$ è definito dalla coppia $(i,j)$). Trascurando le costanti moltiplicative, potremo allora assumere come misura della dimensione della codifica del grafo, al variare del numero dei nodi e degli archi, la funzione $m\log n$; infatti $\log n$ bit sono sufficienti a rappresentare interi positivi e non superiori ad $n$[1]. Nel seguito, per semplicità, oltre alle costanti moltiplicative, trascureremo anche le funzioni sublineari come la funzione logaritmo; diremo allora che $m$ è la lunghezza dell'input per un grafo con $m$ archi. Nelle ipotesi fatte, la misura della lunghezza dell'input non varia se usiamo una codifica in base $b>2$. Se invece si usasse una codifica unaria la lunghezza dell'input nell'esempio in questione diventerebbe $nm$, aumentando considerevolmente.
A questo punto la funzione $g(n)$, introdotta precedentemente, risulta definita in modo sufficientemente rigoroso. In pratica essa continua però ad essere di difficile uso come misura della complessità; infatti risulta difficile se non praticamente impossibile la valutazione di $g(n)$ per ogni dato valore di $n$. Si

(1) In generale, tranne quando sarà esplicitamente detto il contrario, se i dati di un problema sono costituiti da $n$ numeri, considereremo come limitata da $\log n$ il numero di bit necessari per la codifica in binario dei numeri stessi.

risolve il problema sostituendo alla $g(n)$ il suo ordine di grandezza; si parlerà allora di *complessità asintotica*.
Data una funzione $g(x)$, diremo che:
i) $g(x)$ è $O(f(x))$ se esistono due costanti $c_1$ e $c_2$ per cui per ogni $x$ è $g(x) \leq c_1 f(x)+c_2$;
ii) $g(x)$ è $\Omega(f(x))$ se $f(x)$ è $O(g(x))$;
iii) $g(x)$ è $\Theta(f(x))$ se $g(x)$ è allo stesso tempo $O(f(x))$ e $\Omega(f(x))$.

Sia $g(x)$ il numero di operazioni elementari che vengono effettuate da un algoritmo $A$ applicato alla più difficile istanza, tra tutte quelle che hanno lunghezza di input $x$, di un dato problema $P$. Diremo che la complessità di $A$ è un $O(f(x))$ se $g(x)$ è un $O(f(x))$; analogamente diremo che la complessità di $A$ è un $\Omega(f(x))$ o un $\Theta(f(x))$ se $g(x)$ è un $\Omega(f(x))$ o un $\Theta(f(x))$.

Nel proporre algoritmi per la soluzione di problemi si punta sempre ad avere una complessità polinomiale nella dimensione dell'input. Nella pratica un algoritmo polinomiale è ritenuto efficiente, a differenza dagli algoritmi esponenziali. Le seguenti tabelle chiariscono il perché di questa distinzione. Nella tabella 1, vengono forniti, per diverse funzioni di complessità, i tempi di esecuzione su un calcolatore che esegue un milione di operazioni elementari al secondo; nella tabella 2, vengono indicati i miglioramenti ottenibili, in termini di dimensioni delle istanze risolvibili, per diverse funzioni di complessità, al migliorare della tecnologia dei calcolatori; con $x_i$ abbiamo indicato la dimensione di un'istanza risolvibile oggi in un minuto per la $i$-esima funzione di complessità.

| | **Dimensioni dell'istanza** | | | |
|---|---|---|---|---|
| **Funzione** | $n$=10 | $n$=20 | $n$=40 | $n$=60 |
| $n$ | $10^{-5}$ sec. | $2\times10^{-5}$ sec. | $4\times10^{-5}$ sec. | $6\times10^{-5}$ sec. |
| $n^3$ | $10^{-3}$ sec. | $8\times10^{-3}$ sec. | $6.4\times10^{-2}$ sec. | $2.2\times10^{-1}$ sec. |
| $n^5$ | $10^{-1}$ sec. | 3.2 sec. | 1.7 min. | 13 min. |
| $2^n$ | $10^{-3}$ sec. | 1 sec. | 12.7 giorni | $36.6\times10^3$ anni |
| $3^n$ | $6\times10^{-2}$ sec. | 1 ora | $4\times10^5$ anni | $1.3\times10^{13}$ anni |

Tabella 1: tempi di esecuzione su un computer da $10^6$ operazioni al secondo al variare della dimensione dell'input

| **Funzione di complessità** | **Computer odierno** | **100 volte più veloce** | **10000 volte più veloce** |
|---|---|---|---|
| $n$ | $x_1$ | $100\ x_1$ | $10000\ x_1$ |
| $n^3$ | $x_2$ | $4.6x_2$ | $21.5x_2$ |
| $n^5$ | $x_3$ | $2.5x_3$ | $6.3x_3$ |
| $2^n$ | $x_4$ | $x_4 + 6.6$ | $x_4 + 13.2$ |
| $3^n$ | $x_5$ | $x_5 + 4.2$ | $x_5 + 8.4$ |

Tabella 2: incremento delle dimensioni dell'input al variare della velocità del computer, a parità di tempo di esecuzione

Si noti come, in tabella 1, al crescere delle dimensioni dell'input, per algoritmi polinomiali il tempo di esecuzione cresce, ma rimane a livelli accettabili, mentre per gli algoritmi esponenziali tale tempo, pur rimanendo competitivo per istanze piccole, diventa presto insostenibile. Tale differenza si nota anche in tabella 2, dove gli algoritmi polinomiali beneficiano dei progressi nella velocità di calcolo aumentando di un fattore moltiplicativo la dimensione dell'input delle istanze risolvibili in un minuto, mentre il progresso per gli algoritmi esponenziali è dato da una costante additiva.

Nel capitolo 2 abbiamo analizzato la complessità di vari algoritmi, alcuni dei quali sono evidentemente polinomiali come ad esempio l'algoritmo per la visita di un grafo, gli algoritmi per la determinazione dell'albero dei cammini minimi, o l'algoritmo dei cammini aumentanti di Edmons-Karp. Un algoritmo palesemente esponenziale è quello di Branch and Bound illustrato in questo capitolo, che nel caso pessimo deve esplorare tutto l'albero di enumerazione che è esponenziale nel numero di variabili.
Nella classificazione degli algoritmi in base alla loro complessità computazionale abbiamo, a dire il vero, una fascia intermedia. Si considerino ad esempio l'algoritmo di eliminazione dei cicli negativi per il flusso di costo minimo o, sempre per lo stesso problema, l'algoritmo dei cammini minimi successivi. Nella valutazione della complessità di tali algoritmi abbiamo un polinomio tra i cui parametri non compare solamente la dimensione dell'input, ma anche un termine relativo ai dati (capacità massima o costo massimo degli archi). Per esempio abbiamo una complessità di $O(nm^2C_{max}U_{max})$ per l'algoritmo di eliminazione dei cicli negativi. Se $C_{max}$ e $U_{max}$ assumono valori particolarmente elevati, la complessità cresce, e se consideriamo che $C_{max}$ e $U_{max}$ sono codificati in binario, la complessità cresce esponenzialmente nella dimensione dell'input. Se però, invece di ricorrere a una codifica binaria, "bariamo" codificando l'input con un codice unario, la complessità è polinomiale nella dimensione dell'input facendo rientrare il nostro algoritmo nella classe di quelli polinomiali.

Viene detto *pseudopolinomiale* un algoritmo che risolva uno di tali problemi in tempo polinomiale nella lunghezza dell'input codificato in "unario".

**Esercizio**
Classificare gli algoritmi visti nel capitolo due secondo la distinzione "polinomiale", "esponenziale" e "pseudopolinomiale". In particolare, in quale classe rientra l'algoritmo di capacity scaling per il flusso massimo?

## *4.2 Problemi trattabili e problemi intrattabili*

Alla luce della classificazione degli algoritmi introdotta in precedenza, basata sulla loro complessità, possiamo tentare di classificare anche i problemi in base agli algoritmi che li risolvono. Chiameremo *trattabili* i problemi per cui esistono algoritmi la cui complessità sia un $O(p(x))$ con $p(x)$ un polinomio nella dimensione dell'input $x$, e *intrattabili* i problemi per cui un tale algoritmo non esiste.
Il punto critico di questo criterio di classificazione sta proprio nell'affermare che per un dato problema non esiste un algoritmo di soluzione polinomiale. Se non siamo in grado di trovare un algoritmo efficiente per un dato problema, quanto è dovuto alla sua reale difficoltà e quanto alla nostra scarsa capacità? La questione rende delicata la classificazione, ed è stata risolta solo nei primi anni 70 con la teoria della complessità computazionale.
Per potere effettuare una più rigorosa classificazione, facciamo riferimento a problemi in forma decisionale, ovvero a problemi che danno come risposta SÌ o NO. Uno di questi problemi è per esempio il problema di soddisfattibilità visto nel capitolo 1, o il problema di connessione di un grafo, visto nel capitolo 2. Si noti che un problema di ottimizzazione può sempre essere ridotto a un problema in forma decisionale. Consideriamo per esempio il problema:

$$P\text{: max } \{ cx\text{: } x \in F\}.$$

Introducendo un parametro $k$, la versione decisionale $P_k$ del problema $P$ è la seguente:

$$P_k : \{ x \in F,\ cx \geq k\} \neq \emptyset\ ?$$

la cui risposta è un SÌ o un NO e per fornire una certificazione della risposta SÌ basta dare una soluzione $x \in F$ tale che $cx \geq k$.

Disponendo di un algoritmo efficiente per risolvere il problema $P_k$, per qualsiasi valore di $k$, possiamo risolvere efficientemente anche il problema $P$. Infatti basta effettuare una ricerca binaria sul valore della soluzione ottima, risolvendo ad ogni passo un problema decisionale $P_k$ e raffinando la ricerca a seconda che la risposta di $P_k$ sia stata positiva o negativa. Nel caso di risposta positiva concentriamo la nostra ricerca su valori maggiori di $k$, in caso contrario andiamo a cercare la soluzione tra quelle a valore minore o uguale di $k$. Se $O(p(n))$ è la complessità dell'algoritmo che risolve $P_k$, con $n$ dimensione dell'input, e $(\underline{K}, \overline{K})$ sono una stima per difetto e una stima per eccesso del valore dalla soluzione ottima di $P$, La complessità di questo algoritmo di ricerca binaria è $O(\log (\underline{K} - \overline{K})\, p(n))$. Pertanto se $p(n)$ è un polinomio in $n$ anche l'algoritmo di ricerca binaria continua ad essere un algoritmo polinomiale.

<u>Esempio: insieme di nodi indipendenti di massima cardinalità</u>
Dato un grafo non orientato $G=(N,A)$, si definisce un insieme di nodi indipendenti un sottoinsieme $S \subseteq N$, tale che per ogni coppia di nodi $i$ e $j$ appartenenti all'insieme indipendente, non esiste il corrispondente arco in $A$. Pertanto

$$ISP\text{: } \max \{ |S| \text{: } \forall i,j \in S,\ (i,j) \notin A \},$$

e la corrispondente versione decisionale, dato un limite $k>0$, è:

$$ISP_k : \{ S \text{: } \forall i,j \in S,\ (i,j) \notin A,\ |S| \geq k \} \neq \emptyset \ ?$$

Una prima importante classe di problemi è la classe **NP**, costituita da tutti i *problemi decisionali certificabili in tempo polinomiale* da una macchina R.A.M.. Questo significa che, data una qualsiasi istanza di un problema decisionale $P$ e una soluzione $x$, esiste un algoritmo polinomiale che controlla se la soluzione data è una risposta SÌ per $P$.

<u>Esempio SAT</u>
Nel capitolo 1 abbiamo introdotto il problema di soddisfattibilità (SAT) che come vedremo ha un ruolo fondamentale nella teoria della complessità computazionale. Data una formula logica in forma normale congiuntiva $A=C_1 \wedge \ldots \wedge C_m$, in cui ogni clausola $C_i$ è una disgiunzione di letterali $(\pm P_j)$, e le proposizioni elementari che compongono la formula sono $P_1, \ldots, P_n$. Il problema consiste nel trovare una assegnazione di valori di verità alle proposizioni elementari $P_1, \ldots, P_n$ che renda vera la formula. Il problema è di tipo decisionale, la dimensione dell'input è $O(mn)$ dato che la formula è composta da $m$ clausole ognuna delle quali contiene al più $n$ letterali. Il certificato di una risposta SÌ è dato dall'assegnazione di valori quindi ha lunghezza $O(n)$, pertanto SAT appartiene a **NP**.

<u>Esempio: grafo connesso</u>
Prendiamo in esame il problema di determinare la connettività di un grafo orientato visto nel capitolo 2. Dato un grafo orientato $G=(N,A)$ e due nodi $s$ e $t$ di $N$, il problema consiste nel dire se esiste un cammino orientato da $s$ a $t$. Il problema è ovviamente di tipo decisionale. Se indichiamo con $n$ e $m$ rispettivamente il numero di nodi e il numero di archi del grafo, la dimensione dell'input è $O(n+m)$. Il certificato di una risposta SÌ è dato dal cammino stesso che collega $s$ a $t$ e che quindi può venire rappresentato da una sequenza di al più $n$-1 archi consecutivi $(s,i_1), (i_1,i_2), \ldots, (i_r,t)$. Pertanto possiamo concludere che il problema è in **NP**. Si noti che in questo caso anche la risposta NO è certificabile polinomialmente, infatti la non esistenza di un cammino tra $s$ e $t$ viene provata dall'esistenza di una

partizione dell'insieme dei nodi $N$ in due sottoinsiemi $N_s$ e $N_t$ tali che $s \in N_s$, $t \in N_t$ e tutti gli archi che hanno estremi a cavallo della partizione hanno la coda in $N_t$ e la testa in $N_s$.

Esempio: insieme di nodi indipendenti

Consideriamo nuovamente il problema dell'insieme di nodi indipendenti di massima cardinalità (*ISP*) introdotto poc'anzi. Una possibile soluzione $S$ è data da un sottoinsieme di nodi, quindi l'input al problema di certificazione, dato dal grafo $G=(N,A)$, dal sottoinsieme $S$ e dalla costante $k$, ha dimensione $O(m+n)$. Per verificare se $S$ è effettivamente una soluzione al problema dobbiamo verificare innanzitutto che $|S| \geq k$, e in seguito, esaminando ogni arco di $A$ in sequenza, possiamo verificare se entrambi gli estremi sono in $S$. Se questo non succede, allora la soluzione data è una risposta SÌ. La complessità di questa operazione è $O(m)$, quindi il problema del massimo insieme di nodi indipendenti in forma decisionale è in **NP**.

**Esercizio**

Come fare a certificare una risposta NO al problema di soddisfattibilità? Si riesce a darne un certificato polinomiale?

Un sottoinsieme della classe **NP** è la cosiddetta classe **P** dei problemi risolubili in *tempo polinomiale*, che contiene tutti quei problemi decisionali per i quali esistono algoritmi di complessità polinomiale che li risolvono (*problemi polinomiali*).

Esempio: grafo connesso

Nel capitolo 2 abbiamo visto che con un algoritmo di visita del grafo a partire dal nodo $s$, in tempo $O(m)$ potevamo dare una soluzione al problema della connessione. Questo significa che il problema appartiene alla classe **P**.

Una domanda particolarmente importante è se esistano problemi in **NP** che possiamo garantire non appartenere anche a **P**, cioè se sia **P≠NP**. Ad esempio il problema SAT e la versione decisionale del problema ISP, per i quali a tutt'oggi non sono ancora stati forniti algoritmi polinomiali di soluzione, sono in **P** e siamo solamente noi a non essere sufficiente bravi a trovare un algoritmo, oppure sono intrinsecamente difficili? A questa domanda nessuno è stato in grado di rispondere, anche si ritiene fortemente probabile che la risposta sia positiva, cioè che sia effettivamente **P≠NP**.

### *4.3 Problemi* **NP***-completi e problemi* **NP***-ardui*

Per cercare di caratterizzare in qualche modo i problemi di **NP** intrinsecamente difficili è stata introdotta un sottoinsieme speciale di problemi. Questa classe, chiamata *classe dei problemi **NP**-completi*, rappresenta il nucleo dei problemi di **NP** ritenuti difficili. Per arrivare a definire formalmente questa classe abbiamo bisogno di introdurre il concetto di riduzione polinomiale che permette di mettere in relazione due problemi.

Molti problemi, anche se in apparenza notevolmente diversi tra loro, possono tuttavia essere ricondotti l'uno all'altro. Dati due problemi decisionali, $P$ e $Q$, diciamo che $P$ si riduce in tempo polinomiale a $Q$, e scriveremo $P \propto Q$, se, supponendo l'esistenza di un algoritmo $A_Q$ che risolva $Q$ in tempo costante (indipendente dalla lunghezza dell'input), esiste un algoritmo che risolve in tempo polinomiale $P$ utilizzando come sottoprogramma $A_Q$. Parliamo in tal caso di *riduzione polinomiale* di $P$ a $Q$.

Esempio: riduzione da ciclo Hamiltoniano a TSP

Facciamo vedere come il problema della esistenza di un ciclo Hamiltoniano (*HC*) su un grafo simmetrico qualsiasi $G=(N,E)$ possa ridursi polinomialmente al problema del commesso viaggiatore (*TSP*) su grafi completi simmetrici e pesati nella sua versione decisionale, cioè determinare l'esistenza di un ciclo Hamiltoniano di peso minore o uguale ad un prefissato valore $k$. Sia $A_{TSP}(N,c,k)$ un algoritmo che, dati un insieme di nodi $N$ e una funzione reale $c : N \times N \rightarrow \boldsymbol{R}$, restituisce SÌ se esiste un insieme di archi costituenti un ciclo Hamiltoniano di costo $\leq k$, e NO altrimenti. Il seguente algoritmo risolve polinomialmente *HC* nell'ipotesi che la complessità di $A_{TSP}(N,c,k)$ sia $O(1)$:

```
Procedure Ciclo_Hamiltoniano(N,E):
   begin
      for all i,j∈N do if (i,j)∈E then c(i,j) := 0 else c(i,j) := 1;
      return A_TSP(N,c,0)
   end.
```

Pertanto possiamo affermare che $HC \propto TSP$. In pratica, questo significa che *HC* non è più difficile da risolvere di *TSP*. Infatti se esistesse un algoritmo polinomiale per *TSP*, quindi $TSP \in \mathbf{P}$, allora anche $HC \in \mathbf{P}$. Viceversa, se riuscissimo a dimostrare che *HC* non ammette algoritmi polinomiali, potremmo dire lo stesso anche per *TSP*. Queste importanti considerazioni sono sintetizzate nella seguente proprietà:

**Proprietà 4.1**

- Se $P \propto Q$ e $Q \in \mathbf{P}$, allora $P \in \mathbf{P}$.
- se $P \propto Q$ e $P \notin \mathbf{P}$, allora $Q \notin \mathbf{P}$;

La relazione $\propto$ ha le seguenti proprietà:

*i*) è riflessiva: $P \propto P$;

*ii*) è transitiva: $P \propto Q$ e $Q \propto R \Rightarrow P \propto R$.

È proprio la transitività della riduzione che ci aiuta nella definizione della classe dei problemi **NP**-completi e nel dimostrare in modo pratico che un problema è **NP**-completo.

**Definizione 4.2**

Un problema $Q$ è detto **NP**-*completo* se $Q \in \mathbf{NP}$ e se per ogni $P \in \mathbf{NP}$ si ha che $P \propto Q$.

Dalla definizione il compito di dimostrare l'**NP**-completezza di un problema potrebbe sembrare insormontabile, in quanto richiederebbe innumerevoli riduzioni. Cook nel 1971 ha risolto questa difficoltà dimostrando che qualsiasi istanza di un problema $P$ di **NP** si trasforma in una opportuna formula logica di lunghezza polinomiale nella dimensione dell'input di $P$. Se ne deduce quindi che qualsiasi problema in **NP** si riduce in tempo polinomiale a *SAT*.

**Teorema 4.3** (Cook, 1971)

Ogni problema $P \in \mathbf{NP}$ si riduce polinomialmente a *SAT* (problema della soddisfattibilità).

Dal teorema di Cook segue che *SAT* è un problema **NP**-completo e costituisce un punto di riferimento fondamentale. La dimostrazione del teorema, piuttosto tecnica, costituisce una pietra miliare nella teoria della computabilità e non dovrebbe mancare nel bagaglio culturale di un buon informatico. Per soddisfare l'eventuale curiosità rimandiamo alla dimostrazione riportata in [Papadimitriou, Steiglitz, 1982].

I problemi **NP**-completi hanno la proprietà che se esiste per uno di essi un algoritmo polinomiale, allora necessariamente tutti i problemi in **NP** sarebbero risolubili polinomialmente, e quindi avremmo che **P** = **NP**. In un certo senso tali problemi sono i più difficili tra i problemi in **NP**.
In figura 23 sono indicate le relazioni tra **NP**, **P** e la classe dei problemi **NP**-completi.

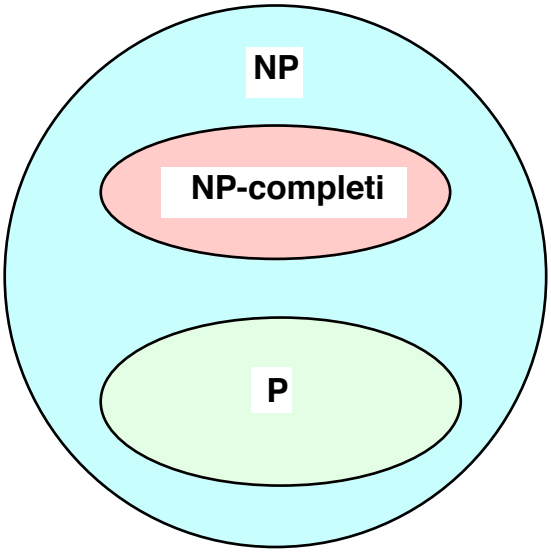


fig. 23: schema della classificazione dei problemi

Una volta dimostrato che *SAT* è **NP**-completo, possiamo dimostrare che un problema *P* appartiene a sua volta alla classe dei problemi **NP**-completi semplicemente riducendo *SAT* a *P* e per la transitività della riduzione otterremo che qualsiasi problema in **NP** si riduce a *P*. Si noti che a questo punto anche *P* diventa utile per la dimostrazione dell'**NP**-completezza di altri problemi. Pertanto, in generale, per dimostrare l'**NP**-completezza di un problema, basta prendere come riferimento un qualsiasi problema **NP**-completo e trovare una opportuna riduzione. Facciamo un esempio trovando una riduzione polinomiale da *SAT* al problema dell'insieme di nodi indipendenti *ISP*.

**Teorema 4.4 [Karp 1972]**
*ISP* è **NP**-completo.

Dim.
In primo luogo, come visto in precedenza, *ISP* è in **NP**, poiché una soluzione di una istanza SÌ è verificabile in tempo polinomiale.
Consideriamo ora una istanza del problema di *SAT*, quindi il problema di determinare se una data formula logica *A* ammette un assegnazione di valori di verità che la rende vera, che sappiamo appartenere alla classe dei problemi **NP**-completi. Per semplicità possiamo assumere che *A* sia in forma normale congiuntiva, quindi specificata da un insieme di clausole $C_1 \wedge \ldots \wedge C_m$, ognuna delle quali è data dalla disgiunzione di letterali (variabili logiche dirette o negate): $C_i = \pm X_1 \vee \ldots \vee \pm X_{i_k}$
Complessivamente, l'insieme delle variabili logiche è dato da $\{X_1, \ldots, X_n\}$.
Ora costruiamo un grafo non orientato $G=(N,E)$ in corrispondenza alla formula *A*. Per ogni clausola introduciamo tanti nodi quanti sono i letterali che la compongono. Introduciamo anche degli archi che collegano tra loro i nodi di ogni clausola, formando quindi tanti sottografi completi $G_i=(N_i,E_i)$ per ogni clausola $C_i$. Il grafo viene completato introducendo degli archi $E'$ che collegano a coppie i nodi relativi alle variabili logiche che compaiono in clausole diverse con diverso segno. Si veda il seguente

esempio in cui dalla formula $A=(X_1 \vee \neg X_2 \vee X_4) \wedge (X_3 \vee \neg X_4) \wedge (\neg X_1 \vee X_2 \vee \neg X_3) \wedge (X_2 \vee \neg X_3)$ si costruisce il grafo in figura 24.

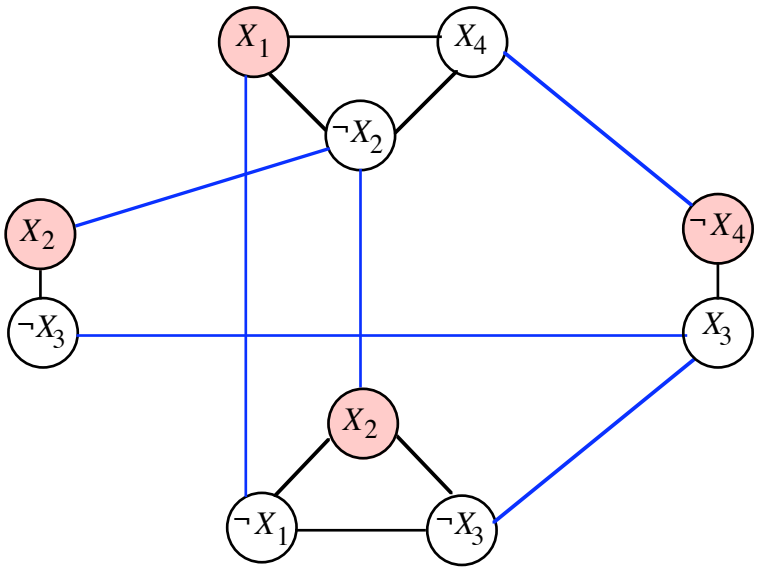


fig. 24: istanza di insieme di nodi indipendenti associato ad una formula logica

Se il grafo così costruito ha un insieme di nodi indipendenti $I$ di cardinalità $m$ abbiamo individuato un insieme di $m$ letterali, appartenenti ognuno a una clausola diversa, compatibili a due a due. Possiamo ottenere un assegnamento di valori di verità come segue. Alle variabili corrispondenti ai nodi in $I$ attribuiamo valore **vero**, se le variabili sono dirette, e valore **falso** se le variabili sono negate, alle eventuali altre variabili, a cui non è stato assegnato alcun valore, attribuiamo un valore arbitrario.
Viceversa se esiste un assegnamento di valori di verità che rende vera $A$, quindi rende vera ciascuna delle $m$ clausole, possiamo costruire un insieme di nodi indipendenti scegliendo un nodo corrispondente a u letterale vero per clausola. ♦

Nell'esempio di figura 24 esiste un insieme di quattro nodi indipendenti associati ai letterali $X_1$ nella prima clausola, $\neg X_4$ nella seconda, $X_2$ nella terza e nella quarta. Un assegnamento di valori di verità relativo a questo insieme di nodi è $X_1 = X_2 =$ **vero**, $X_4 =$ **falso**. Questo assegnamento di valori alle variabili rende vera la formula $A$ per qualsiasi valore di $X_3$.

**Esercizio**
Dato un grafo non orientato $G=(N,E)$, consideriamo il problema di determinare se in $G$ esiste un sottografo completo (detto *clique*) di cardinalità $k$. Dimostrare che detto problema è **NP**-completo utilizzando un tecnica simile a quella utilizzata per *ISP*.

Un problema che abbia come caso particolare un problema **NP**-completo ha la proprietà di essere almeno tanto difficile quanto i problemi **NP**-completi (a meno di una funzione moltiplicativa polinomiale). Un problema di questo tipo si dice **NP**-arduo. Osserviamo che un problema **NP**-arduo può anche non appartenere a **NP**. Ad esempio, sono **NP**-ardui i problemi di ottimo la cui versione decisionale è un problema **NP**-completo. Quindi la versione di ottimizzazione di *ISP* è **NP**-ardua.

Quando si parla di lunghezza dell'input si assume che sia stato usato un modo ragionevolmente compatto per codificare i dati che definiscono il problema. Ad esempio possiamo assumere che se $x$ è la lunghezza del problema con la codifica usata e $y$ la lunghezza con una qualsiasi altra codifica, allora $x$ è un $O(f(y))$, con $f$ un polinomio (possibilmente di grado 1).

Ad esempio il numero intero $a$, se codificato in "binario" ha lunghezza $x = \log a$, mentre ha lunghezza $y = a$ se codificato in "unario". Chiaramente la codifica in "unario" non è accettabile rispetto ai criteri esposti; infatti $y$ cresce esponenzialmente con $x$.
Osserviamo però che esistono problemi che sarebbero risolvibili polinomialmente nella lunghezza dell'input se codificati in "unario", mentre non lo sono con codifiche più compatte. Tali problemi sono in un certo senso più facili di quelli che indipendentemente dalla codifica dell'input richiedono algoritmi esponenziali. Ad esempio, abbiamo visto che il problema dello zaino ammette un algoritmo psudopolinomiale, che fa ricorso alla ricerca di un cammino massimo su un opportuno grafo.
Viene chiamato **NP**-completo *in senso forte* un problema **NP**-completo per cui non esistono algoritmi pseudopolinomiali. Analogamente, viene chiamato **NP**-arduo *in senso forte* un problema di ottimizzazione che abbia come caso particolare un problema **NP**-completo in senso forte.

Un utilissimo strumento per la determinazione di risultati relativi alla complessità computazionale di problemi combinatori è il compendio a cura di P. Crescenzi e V. Kann:
http://www.nada.kth.se/~viggo/wwwcompendium
In tale sito si può trovare una sintetica memoria storica dei risultati di complessità apparsi in letteratura, che viene aggiornata con regolarità, riportando nuovi risultati, nuovi problemi e l'aggiornamento della bibliografia.

## 5 Algoritmi approssimati

Trovandoci di fronte ad istanze di un problema **NP**-arduo, possiamo prevedere che i tempi di soluzione siano elevati. Molto spesso però, il tempo a nostra disposizione per risolverli è limitato e dobbiamo per forza di cose abbassare le nostre pretese accontentandoci di una buona soluzione, non necessariamente ottima, purché ottenuta rapidamente. Inoltre osserviamo che l'ottimalità è molto spesso una proprietà importante dal punto di vista teorico, ma di incerto significato in pratica. Abbiamo già discusso nel capitolo 1 di come ci sia sempre una certa distanza tra realtà e modello. Tale distanza è dovuta anche alla difficoltà (o spesso impossibilità) di conoscere esattamente tutte le grandezze in gioco; pertanto i coefficienti che compaiono nel modello sono affetti da errori spesso considerevoli, alcuni vincoli possono essere stati tralasciati, etc. Ciò contribuisce a rendere in molti casi poco rilevante dal punto di vista pratico la ricerca dell'"ottimo teorico".
In tali casi si fa ricorso ad *algoritmi approssimati*, detti anche *euristiche* perché fanno ricorso a conoscenze della struttura del particolare problema e delle caratteristiche delle soluzioni ottime.
Nel caso di uso di algoritmi euristici è importante (ma non sempre facile, né possibile) effettuare un'analisi dell'errore che si commette accettando la soluzione ottenuta invece di quella ottima.
Si consideri il problema min $\{c(x): x \in F)$.
Sia $A$ un algoritmo per tale problema e $x_A$ la soluzione (non necessariamente ottima) da esso fornita e siano:

$$z_{OPT} = \{\min c(x): x \in F\},$$

$$z_A = c(x_A).$$

Definiamo:

*errore assoluto* $\qquad E_A = z_A - z_{OPT}$

*errore relativo* $$R_A = \frac{z_A - z_{OPT}}{z_{OPT}}$$

Un algoritmo $A$ è *ε-approssimato*, con ε>0, se è:

$$R_A \leq \varepsilon \ .$$

Uno *schema di approssimazione* è un algoritmo approssimato tale che, per ogni ε>0, produce una soluzione con $R_A \leq \varepsilon$. Ovviamente la complessità di un tale algoritmo è una funzione dell'approssimazione ε voluta (crescente con 1/ε).
Uno schema di approssimazione è detto:

- *polinomiale*, se la sua complessità è polinomiale in $n$, dimensione dell'input;
- *pienamente polinomiale*, se la sua complessità è polinomiale in $n$ e in 1/ε.

Una interessante branca della teoria della complessità riguarda l'*approssimabilità* dei problemi. Ovvero, per i problemi **NP**-ardui, determinare classi particolarmente difficili che non ammettono schemi di approssimazione polinomiale.

Come quando abbiamo affrontato la modellazione dei problemi di decisione, anche nel caso delle euristiche è difficile sistematizzare la trattazione e dare una "ricetta" infallibile per la produzione di buoni algoritmi. I migliori algoritmi euristici sono infatti il frutto di esperienza, a volte di fortuna, e della conoscenza delle caratteristiche specifiche dei vari problemi. Vista la difficoltà del compito, che richiederebbe un intero corso, e il poco spazio a disposizione, ci limitiamo a descrivere due schemi algoritmici molto generali, che per altro abbiamo già applicato a vari problemi visti nei precedenti capitoli. La prima classe è quella degli algoritmi *greedy* che costruiscono iterativamente soluzioni ammissibili senza mai tornare sulle scelte fatte. La seconda classe è quella degli algoritmi di *ricerca locale* che, a partire da una soluzione ammissibile, cercano possibili soluzioni migliori ottenute apportando piccole modifiche alla soluzione data. I due tipi di algoritmo possono venire utilizzati in sequenza, ricorrendo ad un greedy per generare una soluzione ammissibile, che successivamente si tenta di migliorare con un algoritmo di ricerca locale.

## *5.1 Algoritmi greedy*

Prima di dare la descrizione formale dello schema degli algoritmi di tipo greedy, analizziamo un semplice esempio e tentiamo di costruirne una soluzione basandoci sul "buon senso".

<u>Esempio: algoritmo costruttivo per il problema dello zaino</u>
Consideriamo l'istanza del problema dello zaino dell'esempio del rapinatore del capitolo 1:

$$\begin{aligned} \max \quad & 7x_1 + 2x_2 + 4x_3 + 5x_4 + 4x_5 + x_6 \\ & 5x_1 + 3x_2 + 2x_3 + 3x_4 + x_5 + x_6 \leq 8 \\ & x_1,\ x_2,\ x_3,\ x_4,\ x_5, x_6 \in \{0, 1\}. \end{aligned}$$

Una soluzione presumibilmente buona, anche se in generale non ottima, si può ottenere procedendo secondo il seguente scema informale. Si ordinano gli indici degli oggetti in ordine di profitto unitario decrescente; quindi, si esaminano le variabili seguendo l'ordine stabilito: a ciascuna variabile si assegna un valore 1 se l'aggiunta dell'oggetto non viola il vincolo di capacità, altrimenti le si assegna il valore 0 e, di fatto, si scarta definitivamente l'oggetto corrispondente.
Nel nostro caso, i passi che il metodo descritto farebbe sono i seguenti:

1. L'oggetto con costo unitario maggiore è il quinto, il cui rapporto costo/peso è pari a 4; poiché l'oggetto ha peso 1 e la capacità residua dello zaino al momento è 8, si pone $x_5$=1, lasciando posto nello zaino per altre 7 unità di peso.
2. Il successivo oggetto nell'ordine scelto è il terzo il cui costo unitario è 2; esso ha un peso minore della capacità residua, quindi si pone $x_3$=1; la capacità residua dello zaino scende di conseguenza a 5.
3. Il successivo oggetto è il quarto, il cui costo unitario è 5/3; anche in questo caso, disponendo di sufficiente capacità possiamo porre $x_4$=1; lo zaino rimane con una capacità residua di 2 unità.
4. L'oggetto successivo è il primo con costo unitario 7/5; esso ha però un peso (5) superiore alla capacità residua dello zaino (2) e la variabile corrispondente $x_1$ viene posta a zero.
5. L'oggetto successivo è il sesto che ha costo unitario 1; poiché il suo peso è 1, possiamo porre $x_6$=1; lo zaino rimane con una capacità residua di 1 unità.
6. L'ultima oggetto considerato è il secondo che, avendo un peso superiore alla capacità residua dello zaino, viene scartato ponendo a zero la variabile $x_2$.

La soluzione ottenuta è allora (0, 0, 1, 1, 1, 1), con costo 14 e peso totale 7. È facile vedere che questa soluzione non è ottima: infatti la soluzione (1, 0, 1, 0, 1, 0), con peso totale 8, ha un costo di 15. L'algoritmo descritto pertanto non garantisce l'ottimalità della soluzione; tuttavia ha il vantaggio di essere di semplice implementazione e computazionalmente molto efficiente, essendo la sua complessità, a meno dell'ordinamento degli oggetti, lineare nella dimensione dell'input.

Un algoritmo di questo tipo viene detto algoritmo *greedy* (vorace). Si tratta di un tipo di algoritmo che determina la soluzione attraverso una sequenza di decisioni parziali (localmente ottime), senza mai tornare, modificandole, sulle decisioni prese. Questi algoritmi sono di facile implementazione e notevole efficienza computazionale, ma, sia pure con alcune eccezioni di notevole rilievo, non garantiscono l'ottimalità, e a volte neppure la ammissibilità. La classe dei problemi che vengono risolti all'ottimo dagli algoritmi greedy è detta la classe dei *matroidi*. Per approfondire la conoscenza dei matroidi si rimanda al libro di Lawler.

Per introdurre in modo formale questi algoritmi solitamente si fa riferimento a problemi rappresentati sotto forma di sistemi di insiemi indipendenti.
Individuiamo un insieme $E$ (*ground set*) che può essere inteso come l'insieme degli elementi che compongono le soluzione del problema sotto esame. Nel caso del problema dello zaino, l'insieme $E$ è dato dall'insieme degli oggetti da scegliere. Inoltre si individua l'insieme $F$ dato da una famiglia di sottoinsiemi di $E$ ($F \subseteq 2^E$), tale che valga la proprietà di chiusura per inclusione:

$$(A \subseteq B) \text{ e } (B \in F) \Rightarrow A \in F.$$

L'insieme $F$ rappresenta l'insieme di tutte le soluzioni parziali ammissibili per il problema. Allora chiameremo:

$F$ una *famiglia di insiemi indipendenti,*
$(E,F)$ un *sistema di insiemi indipendenti.*

Un insieme $I \in F$ è detto un *insieme massimale* se:

$$e \in E \setminus I \rightarrow I \cup \{e\} \notin F.$$

Data una funzione

$c: 2^E \rightarrow \boldsymbol{R}$,

chiameremo *problema associato* a $(E,F)$ il seguente problema di ottimizzazione:

Determinare un insieme $I \in F$ massimale, tale che sia:
$c(I) \geq c(Y)$, $\forall\ Y \in F$ e massimale.

Analogamente possiamo far corrispondere a $(E,F)$ un problema di minimo.

Sia dato il problema di ottimo associato al sistema di insiemi indipendenti $(E,F)$ con funzione obiettivo $c$, un algoritmo di tipo greedy costruisce l'insieme $X \in F$, soluzione del problema, partendo dall'insieme vuoto ed inserendovi ad ogni passo l'elemento di $E$ più "promettente" fra quelli che non violano l'indipendenza dell'insieme.
Nella sua forma più generale un algoritmo di questo tipo è descritto dalla seguente procedura **greedy**:

```
Procedure greedy:
  begin
    X := Ø;  S := E;
    repeat
      e := Best(S);  S := S \ {e};
      if Ind(X,e) then X := X ∪{e}
    until S = Ø
  end.
```

L'algoritmo fa uso delle due sottoprocedure **Best** e **Ind**:

**Best** fornisce il migliore elemento di $S$ sulla base di un prefissato criterio (ad esempio l'elemento di costo minimo nel caso del problema di albero di copertura di costo minimo, o nel caso di problema di zaino l'elemento con il massimo rapporto $v_i/p_i$.

**Ind** è una funzione logica così definita:

$$\mathbf{Ind}(X,e) = \begin{cases} vero, & \text{se } X \cup \{e\} \in F, \\ falso, & \text{altrimenti.} \end{cases}$$

Nell'esempio dello zaino considerato all'inizio del paragrafo, $F$ è l'insieme di tutti quei sottoinsiemi di elementi il cui peso totale è minore o uguale alla capacità dello zaino (cioè l'insieme di tutte le soluzioni ammissibili).
Analizziamo ora la complessità della procedura **Greedy**.
Sia $|E| = n$, e indichiamo con $h(n)$ la complessità di **Best** e con $k(n)$ la complessità di **Ind**. La complessità dell'intera procedura è allora $O(n(h(n)+k(n)))$. Se supponiamo che gli elementi di $E$ vengano ordinati all'inizio, e che quindi ad ogni passo **Best** fornisca il primo fra i rimanenti, la complessità diventa $O(n\log n+nk(n))$.

<u>Esempio: Set covering</u>
Nel problema di set covering vengono dati un insieme $I$ di elementi e una famiglia $J=\{J_1,\ldots,J_m\}$ di sottoinsiemi di $I$. A ciascun sottoinsieme $I_j$ è associato un costo $c_j$. Si vuole determinare una selezione di sottoinsiemi della famiglia tale che ogni elemento di $I$ compaia in almeno un sottoinsieme scelto e la somma dei costi della scelta fatta sia minima. Tale problema è **NP**-arduo, come si può verificare nel compendio del sito internet menzionato nella sezione relativa alla complessità dei problemi. Formuliamo il problema in termini di sistema di insiemi indipendenti. In questo caso l'insieme $E$ è dato

dai sottoinsiemi della famiglia $J$, o se preferiamo dall'insieme di indici $\{1,\ldots,m\}$. La famiglia di insiemi indipendenti $F$ è dato dall'insieme potenza di $E$ quindi dall'insieme di tutti i possibili sottoinsiemi di indici. Si noti infatti che anche la soluzione $X=E$, seppure probabilmente altamente inefficiente, è ammissibile per il set covering. Per specificare l'algoritmo greedy basta definire i dettagli delle funzioni **Best** e **Ind**. La **Best** può essere guidata dal costo $c_j$, scegliendo il sottoinsieme di costo minimo in $S$, oppure può essere guidata dalla cardinalità del sottoinsieme privilegiando i sottoinsiemi più grandi, oppure un compromesso tra questi due criteri privilegiando i sottoinsiemi il cui costo per elemento è minimo:

$$\mathbf{Best}(S) = \operatorname{argmin}\{c_j : j \in S\}$$
$$\mathbf{Best}(S) = \operatorname{argmax}\{|J_j| : j \in S\}$$
$$\mathbf{Best}(S) = \operatorname{argmin}\{\frac{c_j}{|J_j|} : j \in S\}$$

La funzione **Ind** invece, visto che qualsiasi sottoinsieme di indici costituisce una soluzione parziale ammissibile, restituisce sempre true e non deve applicare alcun filtro. In questo modo otteniamo un algoritmo greedy corretto. Se però vogliamo andare più nel dettaglio ci accorgiamo che l'algoritmo, così come specificato sopra, può produrre soluzioni "ridondanti", cioè in cui vi sono alcuni sottoinsiemi che contengono solo elementi già coperti da altri sottoinsiemi scelti. Per ovviare a questo inconveniente possiamo modificare la **Ind** nel seguente modo:

$$\mathbf{Ind}(X,j) = \begin{cases} vero, & \text{se in } J_j \text{ vi sono elementi di } I \text{ non contenuti in alcun } J_i \text{ per } i \in X, \\ falso, & \text{altrimenti.} \end{cases}$$

Questa **Ind** garantisce il fatto che l'algoritmo greedy per il set covering incrementa, a ogni iterazione, l'insieme di elementi coperti e quindi evita di fatto le soluzioni ridondanti. Si noti che per come abbiamo descritto la funzione **Best**, i criteri utilizzati per scegliere un sottoinsieme sono statici, quindi, come nel caso del problema dello zaino, potremmo ricorrere ad un ordinamento iniziale di tutti i sottoinsiemi in base alla particolare funzione scelta. Osserviamo però che nella seconda e terza definizione della **Best** ci riferiamo alla cardinalità del sottoinsieme. Se invece di considerare la cardinalità in assoluto, andassimo a considerare la cardinalità dell'insieme dei nuovi elementi che verrebbero aggiunti alla soluzione corrente, avremmo l'effetto di evitare le ridondanze, senza dover specializzare la funzione **Ind**, e nel contempo avremmo anche un criterio di scelta più sofisticato. Questo criterio di scelta "dinamico" renderebbe inutile l'ordinamento iniziale e richiederebbe a ogni chiamata della **Best** una scansione completa dell'insieme $S$, comportando un piccolo aggravio dal punto di vista della complessità dell'algoritmo.

L'algoritmo greedy nella sua forma generale può essere applicato a molti problemi senza particolari stratagemmi implementativi, come abbiamo visto per lo zaino e per il set covering. Nel caso di problemi più complessi può valer la pena specializzare il greedy per sfruttare la struttura e le caratteristiche specifiche del problema in esame. Vediamo il caso del problema del commesso viaggiatore.

<u>Esempio: algoritmi greedy per il commesso viaggiatore</u>

Il problema è specificato da un grafo non orientato completo $G=(N,A)$ con costi associati agli archi $c_{ij} \geq 0$. Un modo banale di produrre un algoritmo greedy per il problema del commesso viaggiatore procede applicando le regole generali descritte sopra; quindi, tenendo conto che una soluzione può

essere vista come un sottoinsieme di archi, possiamo definire $E=A$ e $F=\{H\subseteq A$: $H$ è un circuito Hamiltoniamo o un suo sottoinsieme non necessariamente connesso$\}$. La **Best**($S$) si limita a restituire l'arco di costo minimo tra quelli in $S$:

$$\mathbf{Best}(S) = \text{argmin}\{c_{ij}: (i,j)\in S\}$$

e la **Ind** verifica se l'aggiunta di un arco alla soluzione corrente permette di arrivare al termine dell'algoritmo con un circuito Hamilotniano:

$$\mathbf{Ind}(X,(i,j)) = \begin{cases} vero\,, & \text{se } X\cup\{(i,j)\} \text{ non contiene nodi con tre archi incidenti o sottocicli,} \\ falso, & \text{altrimenti.} \end{cases}$$

Questo algoritmo ha il difetto di scegliere subito gli archi migliori per poi dover seguire delle scelte obbligate che possono essere arbitrariamente cattive.
Un'altra versione di algoritmo greedy per il TSP si discosta lievemente dallo schema generale descritto sopra, anche se mantiene la caratteristica di costruire incrementalmente una soluzione ammissibile facendo delle scelte localmente ottime.
L'idea è di partire da una soluzione che costituisce già un circuito, anche se non Hamiltoniano, e di incrementarla ad ogni iterazione aggiungendo il nodo che comporta la deviazione più corta. Sia $(i,j)$ l'arco di costo minimo del grafo, possiamo prendere come soluzione iniziale il circuito $i$-$j$-$i$. A ogni iterazione si considera ogni arco $(h,k)$ del circuito e ogni altro nodo $y$ del grafo non ancora incluso nella soluzione, si calcola il costo complessivo della deviazione: $s_{hky} = c_{hy} + c_{yk} - c_{hk}$, e si sceglie la terna $h,k,y$ che minimizza il valore della deviazione $s_{hky}$ inserendo quindi il nodo $y$ nel punto del circuito più conveniente, quindi tra il nodo $h$ e il nodo $k$. Utilizzando come rappresentazione della soluzione $X$ la sequenza di nodi del circuito, le funzioni che specificano l'algoritmo sono:

$$\mathbf{Best}(S,h,k) = \text{argmin}\{s_{hky} = c_{hy} + c_{yk} - c_{hk},: y\notin X,(h,k)\in X\}$$
$$\mathbf{Ind}(X, y, h, k) = vero$$

dove la **Best** restituisce, oltre all'indice del nodo da inserire, anche la coppia di nodi $(h,k)$, adiacenti del circuito corrente, tra cui inserire il nuovo nodo. La **Ind**, con questo criterio di scelta, diventa superflua. Questo algoritmo è noto in letteratura con il nome di *nearest insertion*.

**Esercizio**
Fornire una struttura dati efficiente per implementare l'algoritmo della nearest insertion.

Nel corso degli anni è stata proposta una miriade di algoritmi costruttivi per il TSP riconducibili ad una logica greedy. Vediamo un ulteriore algoritmo greedy che sfrutta il risultato di uno degli altrettanti numerosi rilassamenti del problema. Si consideri il rilassamento del TSP ottenuto tralasciando i vincoli di eliminazione dei sottocicli. Come visto in precedenza, il problema equivale ad un assegnamento su grafo bipartito e dà come risultato una copertura per circuiti del grafo $G$ su cui è definito il problema del TSP. Se siamo fortunati, otteniamo un unico circuito Hamiltoniano che corrisponde alla soluzione ottima del problema, altrimenti avremo tanti sottocircuiti. L'algoritmo prende in esame questi circuiti e li "cuce" assieme a coppie, fino ad ottenere un unico circuito.

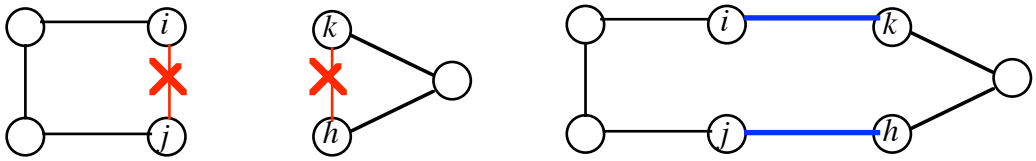


fig. 25: "cucitura" di due circuiti

In figura 25 viene illustrato il procedimento di cucitura di due circuiti. Vengono individuati due archi $(i,j)$ e $(h,k)$ in due circuiti diversi che verranno sostituiti dagli archi $(i,k)$ e $(j,h)$ (oppure $(i,h)$ e $(j,k)$). La nuova soluzione avrà un costo aggiuntivo di $P(i,j,h,k)=c_{ik} + c_{jh} - c_{ij} - c_{hk}$. Gli algoritmi basati su questa tecnica si chiamano algoritmi di "*patching*", talvolta, invece dei costi, vengono utilizzati i costi ridotti frutto della soluzione del problema di assegnamento. È abbastanza semplice arrivare a definire i dettagli di un algoritmo greedy basato sul patching.

**Esercizio**
Specificare la rappresentazione della soluzione parziale e le funzioni **Best** e **Ind** per un algoritmo greedy basato sulla tecnica di patching.

Gli algoritmi greedy, in genere, sono estremamente veloci, ma producono una sola soluzione. Nell'affrontare un problema complesso, talvolta si ha a disposizione abbastanza tempo per tentare di produrre più soluzioni euristiche. Il greedy però non ci viene in aiuto in tal senso, poiché se richiamato più volte sullo stesso input, produce sempre la stessa soluzione. Un modo per ovviare a questo difetto consiste nel rendere non deterministica la scelta della **Best**, in modo che, se l'algoritmo viene richiamato più volte, almeno in linea di principio, ad ogni esecuzione vengono fatte scelte differenti e generate soluzioni alternative. Questa idea sta alla base degli algoritmi GRASP (Greedy Randomly Adaptive Search Procedure). Utilizzando il nostro schema di procedura greedy, l'unica modifica riguarda la funzione **Best**. La nuova funzione, riassunta sotto, riceve in input l'insieme $S$ degli elementi candidati a entrare in soluzione, come nel caso del greedy tradizionale, ma anche un parametro $0<\alpha<1$ che stabilisce una soglia di scostamento dal valore della migliore scelta. La **Best_GRASP** prende in considerazione gli elementi che non si allontanano dal migliore in $S$ per più del $\alpha$% e tra questi ne estrae uno a caso e lo restituisce come risultato.

```
Function Best_GRASP(S,α)
  begin
    v := min (cj: j∈S}
    S' := {j: cj ≥ v (1-α)}        {selezione degli elementi non peggiori di αv rispetto al migliore}
    return (random_selection(S'));
  end.
```

In pratica gli algoritmi GRASP, basati sulla semplice idea riassunta sopra, contengono molti altri "ingredienti" e accorgimenti, e in genere sono affiancati da adeguati algoritmi di ricerca locale. Si invitano gli interessati ad approfondire l'argomento in [], o visitando la pagina web di Mauricio Resende:
http://www.research.att.com/~mgcr/grasp/index.html

## *5.2 Algoritmi di ricerca locale*

Gli algoritmi di ricerca locale prendono in esame una soluzione ammissibile e iterativamente tentano di migliorarla. Durante lo svolgimento del corso abbiamo avuto a che fare con vari algoritmi di ricerca locale. Pensiamo ad esempio agli algoritmi dei cammini aumentanti per il problema del flusso massimo, oppure all'algoritmo di eliminazione di cicli negativi per il problema di flusso di costo minimo, o l'algoritmo del Simplesso Primale-Duale per la programmazione lineare. Questi algoritmi sono caratterizzati dal fatto che ad ogni passo esaminano una soluzione ammissibile, cercandone una migliore tra quelle ad essa "vicine", quindi circoscrivendo la ricerca (da cui il termine "locale") ad un *intorno* della soluzione corrente. Possiamo quindi cercare di dare una descrizione generale di questo paradigma algoritmico per cercare di utilizzarlo come guida per produrre nuovi algoritmi per i problemi più disparati, come abbiamo fatto per il greedy.

Un algoritmo di ricerca locale è in generale caratterizzato dalle seguenti operazioni:
- si determina una soluzione ammissibile di partenza $x^0$;
- si esplora iterativamente una sequenza di soluzioni ammissibili, $x^0, x^1,\ldots, x^k,\ldots$, fermandosi non appena una delle soluzioni trovate viene riconosciuta come una soluzione ottima o non migliorabile.

La ricerca della soluzione che migliora la soluzione corrente $x^i$ viene effettuata in un sottoinsieme delle soluzioni ammissibili vicine a $x^i$ detto *intorno di* $x^i$ e indicato con $I(x^i)$. Generalmente, nel definire un intorno si tengono presente le seguenti regole, ricordando che $F$ indica l'insieme delle soluzioni ammissibili del problema:

1) $x \in F \Rightarrow x \in I(x)$;
2) $x, y \in F \Rightarrow \exists \{x^0, x^1, ..., x^p\} \subseteq F$, con $x^0 = x$, $x^k \in I(x^{k-1})$, $k = 1,\ldots, p$, $x^p = y$.

La prima regola vuole che ogni soluzione appartenga al proprio intorno, questo per garantire una condizione di arresto per l'algoritmo. La seconda, più difficile da verificare, garantisce che date due soluzioni qualsiasi è possibile "migrare" da una all'altra con una sequenza di modifiche compatibili con la definizione di intorno, anche se non necessariamente la sequenza di soluzioni che ne deriva è di miglioramento. Questa regola, almeno in principio, garantisce di poter generare una qualsiasi soluzione a partire da qualsiasi soluzione iniziale. Talvolta però i vincoli del problema in esame sono così stringenti da non garantire questa seconda proprietà per un intorno. Vedremo più avanti come ovviare a questo inconveniente.

L'altro elemento fondamentale di un algoritmo di questo tipo è una funzione $\sigma$, che data una soluzione, $x^k$ fornisce la soluzione successiva $x^{k+1} = \sigma(x^k)$ andandola a cercare all'interno di $I(x^k)$. Tipicamente, in problemi di minimo con funzione obiettivo $c(x)$, si ha che :

$$\sigma(x^k) = \text{argmin}\{c(x): x \in I(x^k)\}$$

Uno schema generale di algoritmo di ricerca locale è il seguente.

```
Procedure Ricerca_Locale(F,c,soluzione):
    begin
      x := Ammissibile(F);
      while σ(x) ≠ x do x := σ(x);
      soluzione := x
    end.
```

Si ha $x=\sigma(x)$ quando la soluzione corrente $x$ viene riconosciuta come ottima dall'algoritmo. Ciò non vuole dire necessariamente che la soluzione sia realmente quella ottima, ma semplicemente che soddisfa un prefissato criterio di arresto.

Se la soluzione determinata dall'algoritmo coincide effettivamente con la soluzione ottima del problema, parliamo di *ottimo globale*, in caso contrario abbiamo un *ottimo locale*. Se l'intorno utilizzato per un determinato problema garantisce sempre di raggiungere l'ottimo globale allora diciamo che l'intorno è *esatto*.

È evidente che più è grande l'insieme $I(x)$ e più alta è la probabilità che l'intorno sia esatto, ma sarà anche più oneroso il calcolo della $\sigma(x)$. Portando il discorso all'estremo potremmo avere $I(x)=F$, ma $\sigma(x)$ sarebbe equivalente a risolvere il problema originale. D'altra parte un intorno più limitato assicura probabilmente un calcolo efficiente della $\sigma(x)$, ma può comportare una convergenza dell'algoritmo in un maggior numero di passi.

Vediamo la definizione di alcuni intorni a partire dai problemi che abbiamo già incontrato e per i quali abbiamo studiato un algoritmo di ricerca locale.

Esempio: flusso massimo

Gli algoritmi dei cammini aumentanti hanno la caratteristica di passare da una soluzione $x^i$ ad una soluzione $x^{i+1}$ variando il flusso lungo un cammino del grafo residuale. Quindi possiamo definire:

$$I(x) = \{x' = x \oplus x_p\text{: } p \text{ cammino da } s \text{ a } t \text{ in } G_R(x)\},$$

dove l'operatore $\oplus$ riassume l'operazione di variazione di flusso lungo il cammino $p$ che comporta un aumento di flusso lungo gli archi di $A^+(x)$ in $p$ e una diminuzione lungo quelli di $A^-(x)$ in $p$.
Gli algoritmi si differenziano a seconda della scelta operata dalla $\sigma(x)$ che può essere: un cammino qualsiasi (algoritmo generico), un cammino con il minimo numero di archi (algoritmo di Edmons-Karp), un cammino con capacità superiore a una certa soglia (algoritmo di capacity scaling).

Esempio: flusso di costo minimo

L'algoritmo di eliminazione dei cicli negativi ha le caratteristiche di un algoritmo di ricerca locale. In questo caso, a differenza del problema di flusso massimo, la definizione di intorno è un po' più delicata, infatti si tratta di preservare l'ammissibilità dei vincoli di conservazione di flusso ai nodi, cosa possibile se si effettuano variazioni di una uguale quantità di flusso lungo dei cicli del grafo residuale. Utilizzando la stessa notazione dell'esempio precedente possiamo definire:

$$I(x) = \{x' = x \oplus x_c\text{: } c \text{ ciclo in } G_R(x)\}.$$

La scelta effettuata dalla $\sigma(x)$ è quella di individuare un ciclo di costo negativo in $G_R(x)$. Questa scelta può essere ulteriormente specializzata: per esempio il ciclo di costo minimo, il ciclo il cui prodotto costo per capacità residua è minimo, il ciclo di costo medio (costo del ciclo diviso il numero di archi) minimo.

Esempio: Programmazione Lineare

Anche l'algoritmo del Simplesso Primale-Duale visto per risolvere i problemi di programmazione lineare della forma max $\{cx\text{: } Ax \le b\}$ ha le caratteristiche di un algoritmo di ricerca locale. Infatti parte da una soluzione ammissibile e si sposta verso soluzioni di miglioramento. Nel caso di questo algoritmo la definizione di intorno $I(x)$ della soluzione corrente è articolato dato che, almeno nella descrizione che ne abbiamo dato, si decompone in due fasi: la ricerca della direzione di spostamento e la determinazione del passo di spostamento lungo la direzione data. Volendo possiamo darne la seguente interpretazione, guardando la figura 26.

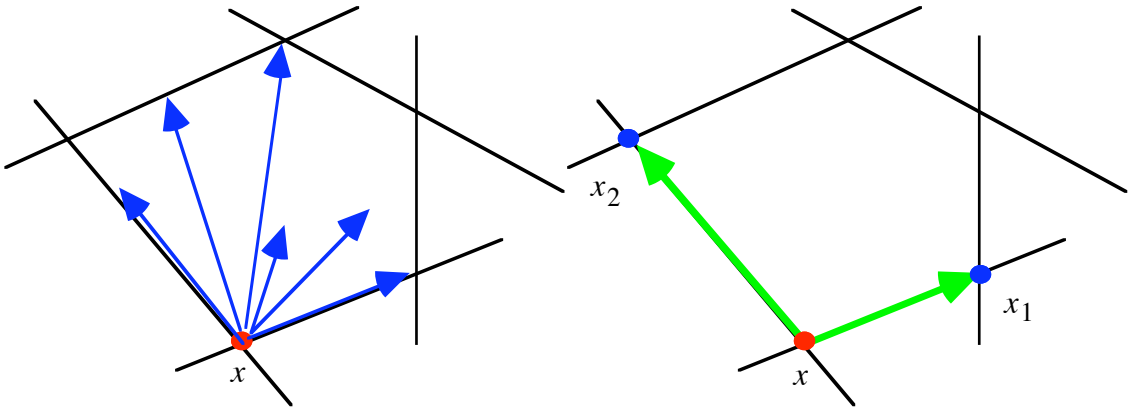


fig. 26: intorno utilizzato dal Simplesso Primale-Duale. Versione estesa e versione limitata

Quindi formalmente possiamo dire:

$$I(x) = \{x' = x + \lambda\xi:\ A_I\xi \leq 0,\ Ax \leq b\}$$

dove $I$ indica, come al solito, l'insieme degli indici dei vincoli attivi in $x$. Se applichiamo in modo esteso la definizione di intorno vediamo che dal punto $x$ possiamo raggiungere in un solo passo qualsiasi punto della regione ammissibile. Questo rende di fatto equivalente il problema originale a quello che risolviamo per determinare la migliore soluzione dell'intorno, ricavando la $\sigma(x)$. In pratica quindi limitiamo l'intorno e la $\sigma(x)$, tra tutte le direzioni, ne sceglie una che resta perpendicolare a tutti i vincoli attivi tranne uno. Inoltre la $\sigma(x)$ restituisce un punto estremo, ottenuto spostandosi lungo la direzione fino a incontrare un nuovo vincolo attivo, se in quella direzione la regione ammissibile risulta essere limitata. In questo modo il numero di soluzioni nell'intorno è finito. Nel caso dell'algoritmo illustrato nel capitolo 3, la scelta è della direzione è fatta con la regola anticiclo di Bland.

<u>Esempio: SAT</u>

Il problema di soddisfattibilità, come abbiamo visto, è un problema decisionale, quindi richiede semplicemente di determinare se la regione ammissibile (insieme delle attribuzioni di valori di verità che rendono vera la formula) sia o meno vuota. Volendo attuare un algoritmo di ricerca locale ci rendiamo conto che già al primo passo, determinando una soluzione ammissibile di partenza avremmo già risolto il problema. Quindi possiamo modificare il problema in uno di ottimizzazione ponendo come funzione obiettivo da minimizzare il numero di clausole non soddisfatte. A questo punto allora qualsiasi attribuzione di valori di verità alle variabili logiche è una soluzione ammissibile. Se abbiamo $n$ variabili, la regione ammissibile è costituita dai vertici dell'ipercubo unitario:

$$F \subseteq \{0,1\}^n$$

Un possibile intorno di una soluzione è definito dal sottoinsieme delle soluzioni che differiscono da quella data per una componente, ovvero che sono a distanza di Hamming minore o uguale a1 rispetto alla soluzione data:

$$I(x) = \{y \in F:\ \sum_i |x_i - y_i| \leq 1\}.$$

In figura 27 è rappresentato il caso di $n = 3$, mettendo in evidenza l'insieme delle soluzioni nell'intorno di (1,0,1): $I(1,0,1) = \{(1,0,1), (1,0,0), (1,1,1), (0,0,1)\}$.

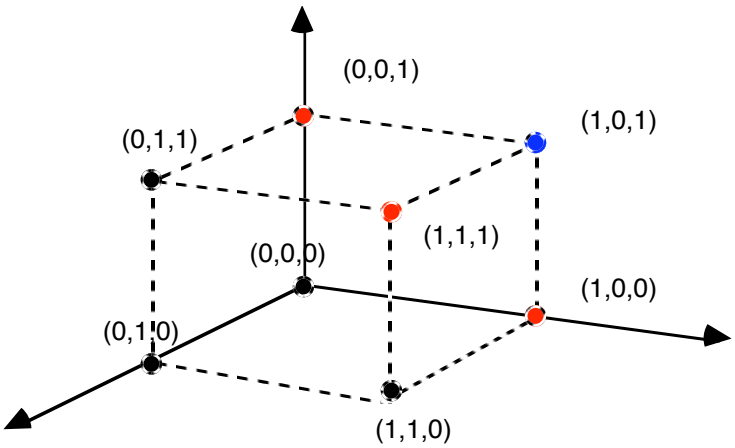


fig. 27: intorno per il problema di SAT

Esempio: albero di copertura di peso minimo

Consideriamo il problema dell'albero di copertura definito su un grafo non orientato $G=(N,A)$ con pesi sugli archi $w_e \geq 0$, per il quale abbiamo dimostrato nel capitolo 2 che l'algoritmo greedy garantisce l'ottimalità della soluzione. Proviamo adesso a fornirne un algoritmo di ricerca locale. Abbiamo quindi un albero di copertura $T$, non necessariamente ottimo, e vogliamo definire un intorno $I(T)$. Per definire tale intorno ci ispiriamo alle proprietà degli alberi di copertura. In particolare, ricordiamo che se viene aggiunto un arco $e \notin T$ all'albero, si viene a formare un unico ciclo $C(e)$, e se vogliamo tornare ad avere un albero è sufficiente eliminare un qualsiasi arco del ciclo $C(e)$. Possiamo allora definire un intorno nel seguente modo:

$$I(T) = \{T' = T \cup \{e\} \backslash e' \ \forall e' \in C(e)\}$$

Se viene effettuato lo scambio tra l'arco $e \in T$ e l'arco $e' \in C(e)$, il peso della nuova soluzione è dato da:

$$w(T') = w(T) - w_e + w_{e'}$$

Volendo implementare la $\sigma(T)$ individuando la migliore soluzione nell'intorno, dobbiamo esplorare esaustivamente tutti gli scambi possibili tra archi non appartenenti a $T$ e archi che formano il ciclo. Quindi le alternative possibili sono $m$-$n$-1 (numero di archi in $A \backslash T$) per $n$ (il massimo numero di archi nel ciclo. Siccome ogni valutazione del peso della nuova soluzione richiede un numero costante di operazioni, la complessità della $\sigma(T)$ è $O(nm)$.

**Teorema 5.1**

$I(T)$ è un intorno esatto.

Dim.

La dimostrazione si basa su criteri analoghi a quelli utilizzati per dimostrare l'ottimalità dell'algoritmo di Kruskal. Sia $T$ un ottimo locale e $T^* \neq T$ l'albero di peso minimo e supponiamo che $w(T^*) < w(T)$. Dato che entrambi gli alberi hanno lo stesso numero di nodi deve esistere un arco $e \in T^* \backslash T$ per cui $w(e) < w(e')$ per ogni $e' \in T$. Se aggiungiamo $e$ a $T$ si forma un unico ciclo $C$ in cui l'arco $e$ ha peso strettamente inferiore al peso degli altri archi del ciclo. Questo contraddice l'ipotesi che $T$ sia un ottimo locale, e che possa esistere un ottimo globale di peso inferiore all'ottimo locale. ♦

**Esercizio**

Definire un intorno $I(T)$ basato sulla proprietà riguardante l'eliminazione di un arco dall'albero e la formazione di due componenti connesse. Dimostrare che, anche in questo caso, che l'intorno è esatto.

Esempio: il problema del commesso viaggiatore

Nel caso del problema del commesso viaggiatore, definito su un grafo non orientato completo $G=(N,A)$ con costi $c_{ij}$ sugli archi, gli intorni sono vari. Il più semplice è quello che effettua una minima variazione. Non potendo effettuare lo scambio di un solo arco in soluzione con uno che non vi appartiene, dato che si otterrebbe la soluzione di partenza. Infatti, eliminando un arco dal circuito Hamiltoniano vi è un solo modo di ricostruire una soluzione ammissibile che coinciderebbe con la soluzione di partenza. Pertanto la variazione minima coinvolge lo scambio di coppie di archi in soluzione con archi non in soluzione. Gli archi non possono venire scelti in qualsiasi modo, infatti se i due archi eliminati risultassero essere adiacenti, anche in questo caso, l'unico modo di riottenere una soluzione ammissibile riproporrebbe la soluzione di partenza. Siano $H$ il circuito Hamiltoniano di partenza e $(i,j)$ e $(h,k) \in H$ i due archi eliminati. La nuova soluzione $H'$ viene ottenuta scambiando $(i,j)$ e $(h,k)$ con $(i,h)$ e $(j,k)$, come illustrato in figura 28.

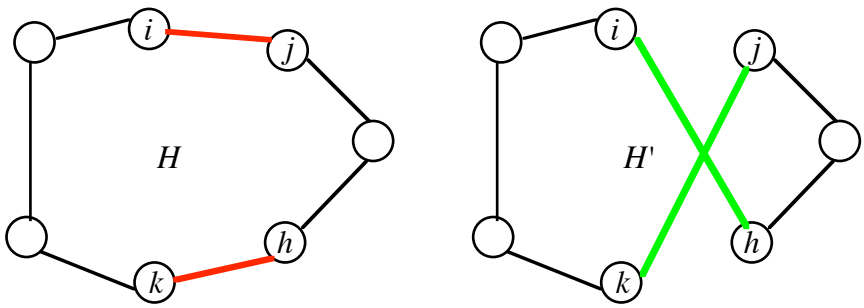


fig. 28: variazione di un circuito Hamiltoniano

Formalmente l'intorno è definito come segue:

$$I(H) = \{H' = H \setminus \{(i,j),(h,k)\} \cup \{(i,h),(j,k)\},\ \forall (i,j),(h,k) \text{ archi non adiacenti di } H\}.$$

Notiamo che a rigori $I(H)$ definito come sopra non contiene $H$ stesso, contravvenendo alla regola i) della definizione di intorno. A questo inconveniente si può sopperire aggiungendo $H$ all'insieme $I(H)$. È facile determinare il numero di soluzioni in un intorno, infatti basta enumerare i modi di scegliere due coppie di archi non adiacenti in un circuito Hamiltoniamo di $n$ archi che ammontano a $n(n\text{-}3)/2$. La determinazione della migliore soluzione dell'intorno, necessaria per valutare la $\sigma(H)$, ha quindi complessità $O(n^2)$, dato che il costo di ogni soluzione $H'$ dell'intorno viene valutata per differenza rispetto al costo della soluzione di riferimento $H$ e richiede un numero costante di operazioni:

$$c(H') = C(H) - c_{ij} - c_{hk} + c_{ih} + c_{jk}.$$

Si osservi che se volessimo considerare una versione orientata del problema del commesso viaggiatore con costi non simmetrici degli archi ($c_{ij} \neq c_{hk}$) il nuovo ciclo Hamiltoniano $H'$, ottenuto secondo la definizione di intorno $I(H)$ data sopra, non rispetta in tutte le parti originali lo stesso orientamento del ciclo $H$. Questo significa che a ogni scambio di coppie di archi $(i,j)$ e $(h,k)$ corrispondono in pratica una coppia di cicli Hamiltoniani (orientati): quello che utilizza gli archi orientati $(i,h)$ e $(j,k)$ percorrendo la porzione di sinistra del ciclo in verso concorde al ciclo di partenza, mentre la porzione di destra in senso inverso (con riferimento alla figura 28); oppure quello che utilizza gli archi orientati $(k,j)$ e $(h,i)$ percorrendo il tratto di sinistra in verso opposto e il tratto di destra in senso concorde rispetto al ciclo originale. Questo significa che per ogni scambio di archi non possiamo procedere per differenza, ma dobbiamo valutare il costo del nuovo ciclo sommando i costi degli archi che lo compongono. La complessità della $\sigma(x)$ diventa allora $O(n^3)$.

**Esercizio**
Si proponga un intorno che preserva l'orientamento delle porzioni di ciclo che rimangono invariate. Valutare la complessità della funzione $\sigma(x)$.

<u>Esempio: il problema dello zaino</u>
Per alcuni problemi la definizione di algoritmi di ricerca locale è poco sistematizzabile. Prendiamo ad esempio il problema di zaino $\max\{cx\colon ax \leq b,\ x_i \in \{0,1\},\ i=1,\ldots,n\}$ e consideriamo la soluzione $x$ definita dalla scelta del sottoinsieme di oggetti $S \subseteq \{1,\ldots,n\}$, cioè $x_i=1$ per ogni $i \in S$. Un intorno della soluzione $S$ potrebbe essere definito eliminando da $S$ un sottoinsieme $K$ di oggetti e sostituendoli con un altro sottoinsieme di oggetti $H$ che vanno a utilizzare la capacità che si è liberata nello zaino. Formalmente:

$$I(S) = \{S' = S \setminus K \cup H\colon K \subseteq S,\ H \subseteq \{1,\ldots,n\} \setminus S \setminus K\}$$

Si noti che la definizione di $I(S)$ è estremamente generale e l'insieme $K$ potrebbe coincidere con tutto $S$. In tal caso una funzione $\sigma(S)$ che va alla ricerca della migliore soluzione dell'intorno coincide con il

problema originale e il paradigma di ricerca locale non ci è di alcun aiuto. Si può allora limitare la cardinalità dell'insieme $K$ per ridurre i gradi di libertà della $\sigma(S)$. Sia quindi $k$ la massima cardinalità ammessa per l'insieme degli oggetti rimossi $K$. Se volessimo vedere come rimpiazzare al meglio gli oggetti rimossi dovremmo risolvere il seguente problema:

$$
\begin{aligned}
P(S)\text{: } \max & \sum_{i \in O} c_i z_i \\
& \sum_{i \in O} a_i z_i \leq b - \sum_{i \in S \setminus K} a_i \\
& z_i \in \{0,1\} \qquad \forall i \in O
\end{aligned}
$$

dove per brevità abbiamo indicato con $O$ l'insieme di oggetti disponibili cioè $O=\{1,\ldots,n\}\setminus S\setminus K$. Quindi, la determinazione di $\sigma(S)$ richiede la soluzione di un problema di zaino, anche se su un numero ridotto di oggetti rispetto a quello originale, e con una capacità inferiore. Questo significa che l'istanza del problema di zaino potrebbe essere sufficientemente ridotto per essere risolto all'ottimo con l'algoritmo basato sulla ricerca di un cammino massimo su grafo aciclico (visto nel capitolo 2). In alternativa, se l'istanza è grande e tenendo conto che va risolta per ogni possibile insieme $K$, si potrebbe pensare di non risolvere all'ottimo il sottoproblema ricorrendo ad un algoritmo di tipo greedy. Avremmo quindi una valutazione euristica della $\sigma(S)$.

**Esercizio**

Definire un intorno basato sull'idea di aggiungere un insieme di oggetti alla soluzione corrente $S$ ed eliminando altri oggetti in modo da non superare la capacità massima dello zaino. Specificare il problema da risolvere per valutare la $\sigma(S)$.

Se un algoritmo di ricerca locale converge rapidamente verso un ottimo locale, potrebbe rimanere tempo a disposizione per esplorare altre zone della regione ammissibile e magari per generare soluzioni migliori. Un modo potrebbe fare ricorso a una procedura di *restart* che genera una nuova soluzione ammissibile (ricorrendo a un GRASP o a una procedura di generazione casuale) e riapplicando la ricerca locale. Altri algoritmi che tentano di superare gli ottimi locali sono il *Simulated Annealing* e la *Tabu Search*, che possono essere visti come varianti della ricerca locale.

Il Simulated Annealing, tenta di riprodurre il comportamento di cristallizzazione di composti chimici che, se raffreddati molto lentamente, convergono verso configurazioni di minima energia. Invece di esplorare esaustivamente l'intorno si sceglie una soluzione a caso nell'intorno. Se la soluzione pescata è migliore di quella corrente la variazione viene accettata, altrimenti se la nuova soluzione è peggiore di quella corrente la variazione viene accettata secondo una legge di probabilità che dipende dalla "temperatura" (un parametro che diminuisce all'aumentare del numero di iterazioni) e dall'entità del peggioramento.

La Tabu Search, nella versione più semplice, è totalmente deterministica. È come una ricerca locale in cui ad ogni iterazione la $\sigma(x)$ restituisce la migliore soluzione dell'intorno. Tale soluzione viene sempre accettata, anche quando comporta un peggioramento. Siccome una tale scelta porterebbe facilmente a ciclare tra poche soluzioni, si mantiene una lista di soluzioni *Tabu*, che vengono eliminate dall'intorno per evitare cicli infiniti. L'implementazione pratica della Tabu Search prevede una serie di particolarità anche molto sofisticate.

Per maggiori dettagli sulle varianti degli algoritmi di ricerca locale si suggerisce di fare riferimento al libro